چکیده

با گسترش روزافزون اینترنت اشیا [1] و تنوع فزاینده پروتکل‌های ارتباطی آن مانند MQTT، CoAP، LoRaWAN، 6LoWPAN و NB-IoT، دسترسی به دانش فنی معتبر و به‌روز در این حوزه به چالشی جدی برای مهندسان، پژوهشگران و دانشجویان تبدیل شده است. منابع این دانش — شامل استانداردهای رسمی OASIS و LoRa Alliance، نشریات NIST، RFCهای IETF، پیش‌چاپ‌های arXiv و مقالات ژورنال‌های MDPI — در وب‌سایت‌های گوناگون، با ساختارهای متفاوت و حجم بسیار زیاد پراکنده‌اند. یک مهندس شبکه برای یافتن پاسخ ساده‌ای مانند «حداکثر طول بسته در CoAP چقدر است؟» ناگزیر است ده‌ها صفحه از RFCها و مشخصات فنی را به‌صورت دستی بازخوانی کند.

این پژوهش یک دستیار پرسش‌وپاسخ هوشمند فارسی‌زبان مبتنی بر معماری RAG [2] طراحی و پیاده‌سازی کرده است که تمامی پردازش‌ها را به‌صورت کاملاً محلی روی سخت‌افزار کاربر انجام می‌دهد و هیچ‌گونه وابستگی به سرویس‌های ابری خارجی ندارد. سیستم از سه لایه اصلی تشکیل شده است: لایه جمع‌آوری داده با خزنده‌ای اختصاصی که از هشت منبع معتبر و رسمی به‌صورت خودکار فایل‌های PDF مرتبط با پروتکل‌های IoT را دانلود می‌کند؛ لایه پردازش دانش با چانکر پاراگراف‌محور که تضمین می‌کند هیچ متنی از بین نمی‌رود و سرویس امبدینگ چندزبانه bge-m3 که متن‌ها را به بردارهای ۱۰۲۴بُعدی نرمال‌شده تبدیل می‌کند؛ و لایه پاسخ‌دهی با مدل زبانی Qwen3-4B که با کوانتایز Q4_K_M روی کارت گرافیک سرو می‌شود و از طریق پایپ‌لاین تول‌کالینگِ مبتنی بر قالب Hermes، خود تصمیم می‌گیرد چه زمانی به دانش‌نامه مراجعه کند.

نوآوری کلیدی این پروژه، معماری «ابزار-محور» [3] آن است که برخلاف RAG کلاسیک، هیچ امبدینگ یا جست‌وجوی خودکاری در ابتدای هر سوال انجام نمی‌دهد؛ بلکه پیام کاربر مستقیماً به مدل ارسال می‌شود و خود مدل زبانی بر اساس محتوای پرسش تصمیم می‌گیرد که آیا به دانش‌نامه نیاز دارد یا خیر. این رویکرد باعث می‌شود برای سوالات عمومی مانند «سلام» یا «تشکر» هیچ درخواستی به سرویس امبدینگ ارسال نشود و منابع محاسباتی بهینه‌تر مصرف شوند. علاوه بر این، پارسر مقاوم سیستم قادر است فراخوانی ابزار را چه در قالب تگ‌دار Hermes و چه در قالب JSON خالیِ بدون تگ تشخیص دهد، که به‌همراه فیلتر استریم StreamMaskFilter، تجربه‌ای روان از نمایش زنده پاسخ توکن‌به‌توکن را در رابط کاربری فراهم می‌کند.

سیستم توسعه‌یافته با رابط کاربری مدرن Streamlit در زبان فارسی و جهت‌نمایی راست‌به‌چپ، مدیریت گفتگوها در پایگاه داده SQLite، کنترل زنده عمق حافظه و تعداد نتایج، و نمایش منابع همراه با امتیاز شباهت، یک تجربه کاربری کامل و حرفه‌ای را ارائه می‌دهد. استقرار کل سیستم با دو کانتینر Docker روی کارت گرافیک NVIDIA و سرویس

[1] Internet of Things — IoT

[2] Retrieval-Augmented Generation

[3] Tool-Call-First

llama.cpp امکان‌پذیر است و کل پایپ‌لاین از زمان دریافت سوال کاربر تا نمایش پاسخ نهایی، کمتر از ۸ ثانیه طول می‌کشد. این پروژه نشان می‌دهد که ترکیب هوشمندانه مدل‌های زبانی کوچک اما کارآمد، امبدینگ‌های چندزبانه، و طراحی پایپ‌لاین ابزار-محور می‌تواند راه‌حلی کاملاً محلی، امن و مقیاس‌پذیر برای دستیارهای تخصصی در حوزه‌های فنی مانند IoT ارائه دهد.

کلمات کلیدی: بازیابی افزوده به تولید، مدل‌های زبانی بزرگ، تول‌کالینگ، امبدینگ چندزبانه، اینترنت اشیا، پایگاه داده برداری، فونت بی‌زار، استریم‌لیت، llama.cpp، Qwen3، bge-m3، FAISS.

فهرست مطالب

چکیده ب

۱ مقدمه ۱

۱.۱ زمینه و انگیزه ۱

۲.۱ مشکل اساسی و نیاز به سیستم ۲

۳.۱ اهداف پروژه ۳

۴.۱ ساختار گزارش ۳

۲ مرور ادبیات و پیشینه پژوهش ۵

۱.۲ مقدمه ۵

۲.۲ مدل‌های زبانی بزرگ و معماری Transformer ۵

۳.۲ سیستم‌های بازیابی افزوده به تولید (RAG) ۶

۱.۳.۲ Late Chunking ۶

۲.۳.۲ GraphRAG و LightRAG ۶

۴.۲ سیستم‌های امبدینگ متن ۷

۱.۴.۲ مدل bge-m3 . . . . . ۷

۵.۲ پایگاه‌های داده برداری ۷

۶.۲ Tool Calling و Function Calling ۸

۷.۲ اکوسیستم پروتکل‌های IoT ۸

۱.۷.۲ پروتکل MQTT ۸

۲.۷.۲ پروتکل CoAP ۹

۳.۷.۲ LoRaWAN و LPWAN ۹

۴.۷.۲ شبکه‌های 6LoWPAN ۹

۵.۷.۲ پروتکل Matter ۹

۶.۷.۲ ملاحظات امنیتی IoT ۹

۸.۲ چارچوب‌های توسعه RAG و ابزارهای موجود ۱۰

۹.۲ جمع‌بندی ۱۰

۳ معماری کلی سیستم ۱۱

۱.۳ مقدمه ۱۱
۲.۳ نمای کلی معماری ۱۱
۳.۳ انتخاب مدل‌ها و فناوری‌ها ۱۲
۱.۳.۳ انتخاب Qwen3-4B به‌عنوان LLM ۱۲
۲.۳.۳ انتخاب bge-m3 به‌عنوان امبدینگ ۱۳
۳.۳.۳ انتخاب FAISS به‌عنوان ذخیره‌گاه برداری ۱۳
۴.۳ استراتژی Tool-Call-First ۱۳
۵.۳ سناریوی کامل از پرسش تا پاسخ ۱۴
۶.۳ جمع‌بندی ۱۵

٤ جمع‌آوری داده‌ها (Crawler) ١٦

۱.۴ مقدمه ۱۶
۲.۴ منابع پشتیبانی‌شده ۱۶
۳.۴ الگوی طراحی Strategy و کلاس پایه ۱۷
۴.۴ کلمات کلیدی و فیلتر موضوعی ۱۸
۵.۴ فیلتر سال انتشار ۱۹
۶.۴ اعتبارسنجی PDF ۱۹
۷.۴ قابلیت Resume ۱۹
۸.۴ نمونه: کلاس ArxivSource ۲۰
۹.۴ Politeness و مدیریت نرخ ۲۱
۱۰.۴ عایق‌سازی خطا ۲۲
۱۱.۴ خلاصه اجرای خزنده ۲۲

٥ تبدیل به دیتاست و نرمال‌سازی ۲۳

۱.۵ مقدمه ۲۳
۲.۵ فرمت دیتاست CSV ۲۳
۳.۵ استخراج متن از PDF ۲۴
۴.۵ نرمال‌سازی متن ۲۴
۵.۵ پشتیبانی از ستون‌های با نام‌های مختلف ۲۵
۶.۵ آمار دیتاست نهایی ۲۵

٦ تکه‌تکه‌سازی (Chunking) ٢٦

۱.۶ مقدمه ۲۶
۲.۶ الگوریتم چانکینگ پاراگراف‌محور ۲۶
۳.۶ سقف کاراکتر هر چانک ۲۷
۴.۶ نمونه کد چانکر ۲۷

۵.۶ راستی‌آزمایی پوشش کامل ۲۸
۶.۶ گزارش آماری چانکینگ ۲۹
۷.۶ تنظیمات پیش‌فرض چانکینگ ۲۹
۸.۶ جمع‌بندی ۳۰

۷ سرویس امبدینگ (bge-m3) ۳۱
۱.۷ مقدمه ۳۱
۲.۷ معرفی bge-m3 . . . . . ۳۱
۳.۷ چرا bge-m3؟ ۳۱
۴.۷ استقرار bge-m3 روی llama.cpp ۳۲
۵.۷ پردازش دسته‌ای و نرمال‌سازی L2 . . . . . ۳۳
۶.۷ تلاش مجدد با Backoff نمایی ۳۵
۷.۷ کش دیسکی برای Resume ۳۶
۸.۷ حذف متن‌های تکراری ۳۷
۹.۷ آمار عملکرد ۳۸

۸ ذخیره‌گاه برداری FAISS ۳۹
۱.۸ مقدمه ۳۹
۲.۸ انتخاب نوع ایندکس: IndexFlatIP ۳۹
۳.۸ ساخت و ذخیره ایندکس ۴۰
۴.۸ ذخیره روی دیسک ۴۱
۵.۸ بارگذاری از دیسک ۴۱
۶.۸ جست‌وجوی معنایی ۴۲
۷.۸ آمار ایندکس ۴۳
۸.۸ مقایسه با سایر پایگاه‌های داده برداری ۴۴

۹ استقرار با Docker و llama.cpp ۴۵
۱.۹ مقدمه ۴۵
۲.۹ معماری دو کانتینری ۴۵
۳.۹ کانتینر مدل زبانی Qwen3-4B ۴۵
۱.۳.۹ چرا کوانتایز Q4_K_M؟ ۴۷
۴.۹ کانتینر امبدینگ bge-m3 . . . . . ۴۸
۵.۹ تخصیص GPU با NVIDIA Container Toolkit ۴۸
۶.۹ بررسی سلامت سرویس‌ها ۴۹
۷.۹ رابط API سازگار با OpenAI ۴۹
۸.۹ سرور آزمایشی برای توسعه آفلاین ۴۹

۹.۹ جمع‌بندی ۵۰

۱۰ پایپلاین RAG نسخه ابزار-محور **۵۱**

۱.۱۰ مقدمه ۵۱
۲.۱۰ تفاوت با RAG کلاسیک ۵۱
۳.۱۰ کلاس RAGPipeline ۵۲
۴.۱۰ کلاس PipelineRun ۵۲
۵.۱۰ حلقه مدل-ابزار ۵۵
۶.۱۰ رویدادهای پایپلاین ۵۵
۷.۱۰ مدیریت خطا ۵۶
۸.۱۰ حداکثر فراخوانی ابزار ۵۶

۱۱ سیستم Tool Calling و Function Calling **۵۷**

۱.۱۱ مقدمه ۵۷
۲.۱۱ تعریف ابزارها ۵۷
۳.۱۱ آموزش قالب Hermes به مدل ۵۸
۴.۱۱ پارسر مقاوم `parse_tool_calls` ۶۰
۵.۱۱ فیلتر استریم StreamMaskFilter ۶۱
۶.۱۱ کلاس ToolRegistry ۶۲
۷.۱۱ ابزار `search_knowledge_base` ۶۳
۸.۱۱ چرا بدون jinja-- کار می‌کند؟ ۶۴

۱۲ رابط کاربری Streamlit **۶۵**

۱.۱۲ مقدمه ۶۵
۲.۱۲ چیدمان کلی رابط کاربری ۶۵
۳.۱۲ فونت Vazirmatn باندل‌شده ۶۵
۴.۱۲ استایل مدرن و RTL ۶۶
۵.۱۲ مدیریت گفتگوها در SQLite ۶۸
۶.۱۲ تنظیمات زنده ۶۹
۷.۱۲ وضعیت دانش‌نامه و منابع ۶۹
۸.۱۲ نوار وضعیت زنده ۷۰

۱۳ سناریوی کامل از پرسش تا پاسخ **۷۱**

۱.۱۳ مقدمه ۷۱
۲.۱۳ سناریو: پرسش در مورد MQTT 5.0 . . . . . ۷۱
۳.۱۳ مرحله ۱: دریافت و ذخیره پیام کاربر ۷۱
۴.۱۳ مرحله ۲: ساخت پایپلاین و شروع استریم ۷۱

۵.۱۳ مرحله ۳: ارسال پیام به مدل . . . ۷۲
۶.۱۳ مرحله ۴: تصمیم مدل برای فراخوانی ابزار . . . ۷۲
۷.۱۳ مرحله ۵: تشخیص و اجرای ابزار . . . ۷۲
۸.۱۳ مرحله ۶: بازگشت نتیجه به مدل . . . ۷۲
۹.۱۳ مرحله ۷: تولید پاسخ نهایی . . . ۷۲
۱۰.۱۳ مرحله ۸: ذخیره و نمایش منابع . . . ۷۳
۱۱.۱۳ زمان‌بندی کلی . . . ۷۳

۱۴ ارزیابی و تحلیل نتایج ۷۴
۱.۱۴ مقدمه . . . ۷۴
۲.۱۴ معیارهای ارزیابی . . . ۷۴
۳.۱۴ مقایسه Qwen3-4B با مدل‌های جایگزین . . . ۷۴
۴.۱۴ کیفیت پاسخ‌ها . . . ۷۵
۵.۱۴ زمان پاسخ‌دهی . . . ۷۵
۶.۱۴ نقاط قوت سیستم . . . ۷۶
۷.۱۴ نقاط ضعف و محدودیت‌ها . . . ۷۶
۸.۱۴ تحلیل هزینه–فایده . . . ۷۶

۱۵ نتیجه‌گیری و کاربردهای آینده ۷۸
۱.۱۵ خلاصه دستاوردها . . . ۷۸
۲.۱۵ دستاوردهای فنی . . . ۷۸
۳.۱۵ کاربردهای عملی . . . ۷۹
۱.۳.۱۵ مدیریت دانش سازمانی . . . ۷۹
۲.۳.۱۵ دستیار فنی مهندسان . . . ۷۹
۳.۳.۱۵ آموزش و تحقیق . . . ۷۹
۴.۱۵ پیشنهادات توسعه . . . ۷۹
۱.۴.۱۵ Graph RAG و LightRAG . . . ۷۹
۲.۴.۱۵ چانکینگ هوشمند با Late Chunking . . . ۷۹
۳.۴.۱۵ Fine-tune مدل Embedder . . . ۸۰
۴.۴.۱۵ پشتیبانی از تصویر و جدول . . . ۸۰
۵.۴.۱۵ استقرار تولیدی . . . ۸۰
۶.۴.۱۵ ارزیابی خودکار کیفیت پاسخ . . . ۸۰
۵.۱۵ نتیجه‌گیری نهایی . . . ۸۰

آ ساختار پروژه و متغیرهای محیطی ۸۱
آ.۱ ساختار پوشه‌های پروژه . . . ۸۱

آ.۲ متغیرهای محیطی ۸۲
آ.۳ دستورات اجرا ۸۳

فهرست تصاویر

۱.۳ نمای کلی معماری سیستم با دو کانتینر Docker و پروژه پایتون مرکزی. ۱۲

فهرست جداول

۱.۳ مقایسه مدل‌های زبانی کاندید برای استقرار محلی. ۱۳
۱.۴ منابع پشتیبانی‌شده توسط خزنده و روش دسترسی به آن‌ها. ۱۷
۱.۵ فرمت دیتاست CSV. ۲۳
۱.۶ تنظیمات پیش‌فرض چانکینگ. ۲۹
۱.۸ مقایسه پایگاه‌های داده برداری. ۴۴
۱.۱۰ رویدادهای پایپلاین برای رابط کاربری. ۵۵
۱.۱۴ مقایسه عملکرد مدل‌های زبانی مختلف. ۷۵
۲.۱۴ تحلیل هزینه–فایده در مقایسه با API تجاری. ۷۷
آ.۱ متغیرهای محیطی فایل env.. . . . . ۸۳

فهرست برنامه‌ها

۱.۹ □□□ □□□□□□□□□□ □□ .□□□□□□□□□□ ۴۸
۱.۱۲ □□□□□ □□□□□□□ □□□ □□□□ .□□□□ □□□ □□□□□□□ □□□□□ ۶۶

فصل ۱

مقدمه

۱.۱ زمینه و انگیزه

در دهه گذشته، شاهد ظهور پارادایم جدیدی در فناوری هستیم که اینترنت اشیا [1] نامیده می‌شود و در آن میلیاردها دستگاه فیزیکی از حسگرهای دما تا لوکوموتیوهای قطار، از لامپ‌های هوشمند تا پمپ‌های انسولین، از طریق شبکه‌های ارتباطی متنوع به یکدیگر و به اینترنت متصل می‌شوند. تخمین زده می‌شود که تا سال ۲۰۲۵ بیش از ۲۷ میلیارد دستگاه IoT در سراسر جهان فعال باشند و این حجم عظیم از اتصال، نیازمند پروتکل‌های ارتباطی متنوع و سازگار با محدودیت‌های سخت‌افزاری، انرژی و پهنای باند خواهد بود.

اکوسیستم پروتکل‌های IoT به‌طور ذاتی پراکنده و چندسطحی است. در لایه نزدیک به حسگرها، پروتکل‌های کم‌انرژی مانند BLE، Zigbee و Thread با بسته‌های کوچک و نرخ بیت پایین کار می‌کنند. در لایه‌های بالاتر، پروتکل‌های پیام‌محور مانند MQTT، CoAP و AMQP ارتباط بین دستگاه‌ها و سرویس‌های ابری را برقرار می‌کنند. در شبکه‌های LPWAN، فناوری‌هایی مانند LoRaWAN، NB-IoT و Sigfox با تعادل بین برد، مصرف انرژی و نرخ داده، راهکارهایی برای کاربردهای گسترده ارائه می‌دهند. هر کدام از این پروتکل‌ها استاندارد، مشخصات فنی، RFCها و مقالات پژوهشی متعددی دارند که در منابع گوناگون پراکنده‌اند و یک مهندس برای درک کامل حتی یکی از این پروتکل‌ها باید ده‌ها سند فنی را مطالعه کند.

هم‌زمان با این گسترش IoT، انقلاب دیگری در حوزه پردازش زبان طبیعی [2] در جریان است که با معرفی معماری Transformer [۱] آغاز شد و با مدل‌های زبانی بزرگ [3] مانند GPT-3 [۲]، LLaMA [۳]، Mistral [۴] و Qwen3 [۵] به بلوغ رسید. این مدل‌ها با آموزش بر روی میلیاردها کلمه متن، توانایی درک، تحلیل و تولید زبان طبیعی را به شیوه‌ای بی‌سابقه به‌دست آورده‌اند و این امکان را فراهم می‌کنند که به جای جست‌وجوی دستی در اسناد، کاربران با زبان طبیعی و به زبان خودشان (فارسی) سوال بپرسند و پاسخ مستند و دقیق دریافت کنند.

با این حال، استفاده مستقیم از LLMهای عمومی برای پاسخ‌دهی به سوالات تخصصی فنی با چالش جدی مواجه است. اولاً، این مدل‌ها با داده‌های عمومی آموزش دیده‌اند و ممکن است اصطلاحات تخصصی، نسخه‌های جدید استانداردها یا جزئیات فنی خاص پروتکل‌های IoT را به‌درستی ندانند. ثانیاً، پدیده hallucination [۶] که در آن

[1] Internet of Things — IoT

[2] Natural Language Processing — NLP

[3] Large Language Models — LLMs

مدل اطلاعات نادرست اما به‌ظاهر معتبر تولید می‌کند، در حوزه‌های تخصصی خطرناک است؛ چرا که کاربر فنی ممکن است به پاسخ اشتباه اعتماد کند و در طراحی سیستم‌های بحرانی خطا بروز کند. ثالثاً، ارسال سوالات و اسناد داخلی سازمان به APIهای ابری خارجی مانند OpenAI یا Anthropic از منظر حریم خصوصی و امنیت داده قابل پذیرش نیست، به‌ویژه وقتی اسناد حاوی مشخصات اختصاصی محصولات یا طرح‌های مهندسی باشند.

راه‌حل استاندارد برای رفع این چالش‌ها، معماری RAG [۷] است که در آن، به جای اتکای صرف به دانش درون‌ساختی مدل، یک پایگاه دانش خارجی ساخته می‌شود و در زمان پاسخ‌دهی، بخش‌های مرتبط از آن پایگاه بازیابی و در پرامپت مدل قرار می‌گیرند. این رویکرد هم دقت پاسخ‌ها را بالا می‌برد و هم امکان ردیابی منابع را فراهم می‌کند؛ چرا که مدل می‌تواند به‌صورت شفاف به سندی که پاسخ از آن استخراج شده ارجاع دهد. ترکیب این معماری با مدل‌های کوچک اما کارآمد که به‌صورت محلی اجرا می‌شوند، راهکاری کاملاً مستقل از سرویس‌های ابری فراهم می‌آورد که هم از نظر هزینه و هم از نظر امنیت، مزایای قابل توجهی دارد.

۲.۱ مشکل اساسی و نیاز به سیستم

یک مهندس IoT که در حال طراحی یک شبکه حسگر بی‌سیم مبتنی بر LoRaWAN برای پایش خطوط لوله است، در طول یک روز کاری با ده‌ها سوال فنی مواجه می‌شود: «حداکثر طول بسته در LoRaWAN چقدر است؟»، «تنظیمات Spreading Factor برای برد ۱۰ کیلومتر چگونه باید باشد؟»، «تفاوت Class A و Class C در مصرف انرژی چیست؟»، «چگونه SCHC برای فشرده‌سازی هدر IPv6 روی LoRaWAN استفاده می‌شود؟». پاسخ به هر یک از این سوالات نیازمند جست‌وجو در RFCهای مختلف، مشخصات رسمی LoRa Alliance، مقالات IEEE و مستندات فنی است که هر کدام چند ده صفحه طول دارند.

راه‌حل سنتی برای این مشکل، استفاده از موتورهای جست‌وجوی کلیدواژه‌ای مانند Google است که چند مشکل اساسی دارد: اول، خروجی‌ها بر اساس تطبیق کلیدواژه‌ای هستند نه معنایی، بنابراین اگر کاربر عبارت «طول بسته» را جست‌وجو کند، نتایجی که شامل «اندازه payload» یا «MTU» هستند نمایش داده نمی‌شوند. دوم، نتایج جست‌وجو صفحات وب هستند نه پاسخ‌های مستقیم؛ کاربر باید لینک‌ها را باز کند، متن را بخواند و خود پاسخ را استخراج کند. سوم، هیچ تضمینی برای معتبر بودن منابع وجود ندارد و نتایج ممکن است شامل پست‌های وبلاگ غیرتخصصی یا نسخه‌های قدیمی استانداردها باشند.

مدل‌های LLM تجاری مانند GPT-4 یا Claude می‌توانند تا حدی این مشکل را حل کنند، اما سه محدودیت جدی دارند: نخست، مدل‌های تجاری با داده‌های عمومی آموزش دیده‌اند و دانش آن‌ها از پروتکل‌های خاص IoT محدود به آنچه در وب به‌صورت آزاد منتشر شده است می‌باشد. دوم، استفاده از این مدل‌ها نیازمند ارسال داده‌ها به سرورهای خارجی است که از منظر حریم خصوصی، انطباق با مقررات و امنیت داده‌های حساس سازمانی، قابل قبول نیست. سوم، هزینه استفاده از این APIها برای حجم بالای پرسش‌ها (مثلاً در یک تیم مهندسی ده نفره با روزانه صدها پرسش) به‌سرعت قابل توجه می‌شود.

راه‌حلی که این پروژه ارائه می‌دهد، یک سیستم RAG کاملاً محلی است که مدل زبانی Qwen3-4B و امبدینگ bge-m3 را روی سخت‌افزار کاربر سرو می‌کند و به یک پایگاه دانش ساخته‌شده از اسناد رسمی IoT متصل می‌شود. این سیستم نه‌تنها دقت پاسخ‌ها را از طریق بازیابی معنایی بهبود می‌بخشد، بلکه هیچ داده‌ای به خارج از محیط کاربر ارسال نمی‌کند و تمام پردازش‌ها به‌صورت آفلاین انجام می‌شود.

۳.۱ اهداف پروژه

اهداف اصلی این پروژه به شرح زیر هستند:

۱. **طراحی و پیاده‌سازی یک خزنده هوشمند** برای جمع‌آوری خودکار فایل‌های PDF مرتبط با پروتکل‌های IoT از هشت منبع معتبر و رسمی شامل arXiv، RFC Editor، NIST، OASIS، LoRa Alliance، Semantic Scholar، MDPI و IETF Datatracker.

۲. **تبدیل اسناد به یک دیتاست ساختاریافته** با فرمت CSV استاندارد و نرمال‌سازی متون استخراج‌شده از PDFها برای پردازش‌های بعدی، شامل حذف بایت‌های NUL، تطبیق انکودینگ UTF-8 و حذف کاراکترهای زائد.

۳. **پیاده‌سازی چانکر پاراگراف‌محور** که با تضمین پوشش کامل متن، اسناد را به تکه‌های manageable تبدیل می‌کند؛ به‌گونه‌ای که هیچ پاراگرافی از چانک‌ها جا نیفتد و سقف کاراکتر هر چانک رعایت شود.

۴. **سرویس‌دهی امبدینگ چندزبانه** با مدل bge-m3 که قادر است متن‌های فارسی و انگلیسی را به بردارهای ۱۰۲۴بُعدی نرمال‌شده تبدیل کند، با قابلیت دسته‌بندی، نرمال‌سازی L2 و کش دیسکی برای resume در صورت قطع.

۵. **ذخیره‌سازی برداری** با FAISS و IndexFlatIP که امکان جست‌وجوی شباهت کسینوسی با سرعت بالا را بر روی میلیون‌ها بردار فراهم می‌کند.

۶. **پایپلاین RAG نسخه ابزار-محور** که برخلاف RAG کلاسیک، تصمیم به جست‌وجو را به خود مدل می‌سپارد و از طریق Tool Calling با قالب Hermes با مدل ارتباط برقرار می‌کند.

۷. **رابط کاربری مدرن فارسی** با Streamlit که دارای جهت‌نمایی راست‌به‌چپ، مدیریت گفتگوها، تنظیمات زنده و نمایش منابع همراه با امتیاز شباهت است.

۸. **استقرار کامل با Docker** روی کارت گرافیک NVIDIA با دو کانتینر مجزا برای مدل زبانی و سرویس امبدینگ.

٤.۱ ساختار گزارش

در ادامه، ساختار این گزارش به شرح زیر ارائه می‌شود:

- **فصل دوم** به مرور ادبیات و پیشینه پژوهش می‌پردازد؛ شامل مرور مدل‌های زبانی بزرگ، تکنیک‌های RAG، سیستم‌های امبدینگ، پایگاه‌های داده برداری، Tool Calling و اکوسیستم IoT.
- **فصل سوم** معماری کلی سیستم را شرح می‌دهد؛ شامل نمودار لایه‌ای، انتخاب مدل‌ها، استراتژی Tool-Call-First و نمودار سناریوی کامل.

- **فصل چهارم** به ماژول جمع‌آوری داده‌ها (Crawler) اختصاص دارد؛ شامل هشت منبع، الگوی طراحی Strategy، فیلتر کلیدواژه‌ها و اعتبارسنجی PDF.
- **فصل پنجم** فرآیند تبدیل اسناد به دیتاست و نرمال‌سازی را پوشش می‌دهد.
- **فصل ششم** چانکینگ پاراگراف‌محور و تضمین پوشش کامل متن را تشریح می‌کند.
- **فصل هفتم** سرویس امبدینگ bge-m3 را با جزئیات معماری و پردازش دسته‌ای توضیح می‌دهد.
- **فصل هشتم** ذخیره‌گاه برداری FAISS و جست‌وجوی معنایی را بررسی می‌کند.
- **فصل نهم** استقرار با Docker و llama.cpp روی GPU را شرح می‌دهد.
- **فصل دهم** پایپلاین RAG نسخه ابزار-محور را با رویدادها و حلقه تولید پاسخ توضیح می‌دهد.
- **فصل یازدهم** سیستم Tool Calling و پارسر مقاوم را پوشش می‌دهد.
- **فصل دوازدهم** رابط کاربری Streamlit فارسی را شرح می‌دهد.
- **فصل سیزدهم** سناریوی کامل از زمان پرسش کاربر تا پاسخ نهایی را گام‌به‌گام توضیح می‌دهد.
- **فصل چهاردهم** ارزیابی و تحلیل نتایج را ارائه می‌دهد.
- **فصل پانزدهم** نتیجه‌گیری، کاربردهای آینده و پیشنهادات توسعه را شامل می‌شود.

در نهایت، پیوست‌ها شامل ساختار کامل پروژه، متغیرهای محیطی و دستورات اجرا، و فهرست مراجع به سبک IEEE ارائه شده است.

فصل ۲

مرور ادبیات و پیشینه پژوهش

۱.۲ مقدمه

در این فصل، مبانی علمی و فنی پروژه به‌صورت نظام‌مند بررسی می‌شوند تا خواننده با پیشینه پژوهش‌های مرتبط، تکنیک‌های موجود و چالش‌های شناخته‌شده آشنا شود. این مرور شامل شش محور اصلی است: مدل‌های زبانی بزرگ و معماری Transformer، سیستم‌های بازیابی افزوده به تولید (RAG)، تکنیک‌های امبدینگ متن، پایگاه‌های داده برداری، Tool Calling در LLMها، و در نهایت اکوسیستم پروتکل‌های IoT که موضوع دامنه دانش این پروژه است. هر یک از این محورها به‌طور مستقل دارای پژوهش‌های گسترده‌ای است که در ادامه به‌صورت فشرده اما دقیق مرور می‌شوند.

۲.۲ مدل‌های زبانی بزرگ و معماری Transformer

پایه تمامی مدل‌های زبانی مدرن، معماری Transformer است که توسط وسوانی و همکاران [۱] در سال ۲۰۱۷ معرفی شد. ایده کلیدی این معماری، استفاده از مکانیزم Self-Attention است که به هر توکن در یک دنباله، اجازه می‌دهد به تمام توکن‌های دیگر توجه کند و بنابراین وابستگی‌های بلندبرد را به‌صورت موازی (به جای ترتیبی مانند RNNها) محاسبه کند. این امر باعث شد آموزش مدل‌های با میلیاردها پارامتر روی خوشه‌های GPU برای اولین بار امکان‌پذیر شود.

پس از معرفی Transformer، سری مدل‌های GPT [۲] از OpenAI با پارادایم decoder-only و پیش‌بینی توکن بعدی، استاندارد جدیدی را در پردازش زبان طبیعی تعریف کردند. GPT-3 با ۱۷۵ میلیارد پارامتر نشان داد که مدل‌های زبانی بزرگ می‌توانند با few-shot prompting — یعنی با ارائه چند مثال در پرامپت — وظایف جدیدی را بدون آموزش اضافه انجام دهند. این ویژگی که «یادگیری few-shot» نامیده می‌شود، یکی از مهم‌ترین دستاوردهای LLMها محسوب می‌شود.

هم‌زمان با پیشرفت مدل‌های تجاری، جریان متن‌بازی از مدل‌های متن‌باز شکل گرفت که LLaMA [۳] از Meta، Mistral [۴]، Gemma [۸] از Google و خانواده Qwen [۵، ۹] از Alibaba از مهم‌ترین آن‌ها هستند. این مدل‌ها با وزن‌های آزاد منتشر می‌شوند و امکان اجرای محلی روی سخت‌افزار کاربر را فراهم می‌کنند. در این پروژه، از مدل Qwen3-4B استفاده شده است که با ۴ میلیارد پارامتر، تعادل مناسبی بین کیفیت پاسخ و سرعت استنتاج روی کارت

گرافیک مصرفی دارد.

مدل‌های Qwen3 [۵] در نسخه‌های مختلف از ۰.۶ میلیارد تا ۲۳۵ میلیارد پارامتر منتشر شده‌اند و در مجموعه‌داده‌های چندزبانه شامل بیش از ۱۱۹ زبان آموزش دیده‌اند. این مدل‌ها از Grouped-Query Attention برای کاهش مصرف حافظه و از SwiGLU به‌عنوان تابع فعال‌سازی استفاده می‌کنند. نسخه ۴ میلیارد پارامتری، قابلیت اجرای موازی روی یک کارت گرافیک مصرفی با ۸ GB حافظه را دارد و به همین دلیل برای استقرار محلی مناسب است.

۳.۲ سیستم‌های بازیابی افزوده به تولید (RAG)

مفهوم RAG اولین بار توسط لوئیس و همکاران [۷] در سال ۲۰۲۰ به‌صورت رسمی معرفی شد، هرچند ایده ترکیب بازیابی و تولید پیش از آن نیز در سیستم‌های پرسش‌وپاسخ باز دامنه وجود داشت. در معماری RAG پایه، یک مدل زبانی به یک پایگاه دانش خارجی متصل می‌شود؛ به این صورت که هنگام پاسخ به یک سوال، ابتدا بخش‌های مرتبط از پایگاه بازیابی شده، سپس در پرامپت مدل قرار می‌گیرند و مدل بر اساس آن‌ها پاسخ تولید می‌کند. این رویکرد دو مزیت کلیدی دارد: نخست، امکان به‌روزرسانی دانش بدون نیاز به آموزش مجدد مدل؛ و دوم، امکان ردیابی منابع پاسخ که برای شفافیت و اعتمادپذیری حیاتی است.

ایزاکارد و گریو [۱۰] با معرفی روش Fusion-in-Decoder (FiD)، نشان دادند که خواندن هم‌زمان چند سند بازیابی‌شده و ترکیب آن‌ها در فرآیند تولید، می‌تواند به‌طور قابل توجهی دقت پاسخ را افزایش دهد. این کار پایه‌ای برای تکنیک‌های پیشرفته RAG شد که در آن‌ها به جای یک سند، چندین سند بازیابی و در پرامپت قرار می‌گیرند.

مرور جامع گائو و همکاران [۱۱] بر روی RAG نشان می‌دهد که این معماری به سه نسل تقسیم می‌شود: Naive RAG که در آن سوال کاربر مستقیماً امبد شده و جست‌وجو می‌شود؛ Advanced RAG که شامل مراحل پیش‌پردازش، بازنویسی سوال و rerankingاست؛ و Modular RAG که در آن چند ماژول تخصصی (مانند router، selector، evaluator) به‌صورت پویا انتخاب می‌شوند. پروژه حاضر در دسته Modular RAG قرار می‌گیرد؛ چرا که ماژول تصمیم‌گیری برای جست‌وجو به خود مدل سپرده شده است.

۱.۳.۲ Late Chunking

تکنیک Late Chunking[۱۲] که توسط تیم Jina AI معرفی شده، یکی از نوآوری‌های مهم در RAG مدرن است. در chunking سنتی، متن ابتدا به تکه‌های کوچک تقسیم و سپس هر تکه به‌صورت مستقل امبد می‌شود؛ این رویکرد باعث می‌شود اطلاعات زمینه‌ای که در تکه‌های مجاور قرار دارد، در بردار نهایی منعکس نشود. در Late Chunking، ابتدا کل سند به مدل امبدینگ با پنجره زمینه بزرگ داده می‌شود و سپس در لایه نهایی، بردار هر تکه با توجه به موقعیت آن در سند استخراج می‌شود. این کار باعث می‌شود بردار هر تکه، حاوی اطلاعات زمینه‌ای از کل سند باشد و دقت بازیابی افزایش یابد.

۲.۳.۲ GraphRAG و LightRAG

برای پرسش‌هایی که نیاز به درک ارتباطات بین موجودیت‌های مختلف در یک مجموعه اسناد دارند، RAG سنتی کافی نیست. اج و همکاران [۱۳] با معرفی GraphRAG، نشان دادند که ساخت گراف دانش از اسناد (با استخراج

موجودیت‌ها و روابط بین آن‌ها) و سپس خلاصه‌سازی اجتماعات موجودیت‌ها، می‌تواند پاسخ‌گویی به پرسش‌های جامع مانند «موضوعات اصلی در این مجموعه اسناد چیست؟» را ممکن کند. این روش که در Microsoft توسعه یافته، برای مجموعه‌های بزرگ اسناد (۱ میلیون توکن) بهینه شده است.

نسخه سبک‌تر این ایده، LightRAG [۱۴] است که با ساختار گرافی ساده‌تر و الگوریتم بازیابی دو-سطحی (پایین برای موجودیت‌های خاص، بالا برای مفاهیم کلی)، هم دقت بازیابی را بهبود می‌بخشد و هم زمان استنتاج را به‌طور قابل توجهی کاهش می‌دهد. در پروژه حاضر، این تکنیک‌ها به‌عنوان یکی از پیشنهادات توسعه آینده بررسی شده‌اند.

۴.۲ سیستم‌های امبدینگ متن

امبدینگ متن، فرآیندی است که در آن متن به یک بردار عددی با ابعاد ثابت تبدیل می‌شود، به‌گونه‌ای که متون با معنای مشابه، بردارهای نزدیک به هم در فضای برداری داشته باشند. این تبدیل، پایه تمامی سیستم‌های بازیابی معنایی است.

ریمرز و گوریویچ [۱۵] با معرفی Sentence-BERT، نشان دادند که استفاده از شبکه‌های Siamese برای تولید بردار جملات، می‌تواند به‌طور قابل توجهی کارایی بازیابی معنایی را بهبود ببخشد. این کار پایه‌ای برای مدل‌های امبدینگ مدرن مانند bge، E5 و GTE شد.

کارپوخین و همکاران [۱۶] با معرفی Dense Passage Retrieval (DPR)، نشان دادند که بازیابی مبتنی بر بردار، می‌تواند در دقت با بازیابی کلیدواژه‌ای سنتی (BM25) رقابت کند و حتی از آن پیشی بگیرد. DPR از دو مدل BERT مجزا برای پرسش و پاسخ استفاده می‌کند، در حالی که مدل‌های مدرن مانند bge-m3 از یک مدل واحد برای هر دو استفاده می‌کنند.

۱.۴.۲ مدل bge-m3

مدل bge-m3 [۱۷] که توسط تیم BAAI توسعه یافته، یکی از پیشرفته‌ترین مدل‌های امبدینگ چندزبانه است که سه قابلیت کلیدی را در یک مدل واحد ارائه می‌دهد: پشتیبانی از بیش از ۱۰۰ زبان، سه تابع شباهت (شباهت کسینوسی، جست‌وجوی چگال و جست‌وجوی پراکنده)، و پشتیبانی از ورودی‌های با طول متغیر تا ۸۱۹۲ توکن. این مدل با ۵۶۸ میلیون پارامتر و بر پایه معماری XLM-RoBERTa، آموزش دیده است و در معیارهای ارزیابی استاندارد مانند MTEB عملکرد بسیار خوبی نشان داده است.

در این پروژه، bge-m3 به دلیل پشتیبانی هم‌زمان از زبان‌های فارسی و انگلیسی (که در اسناد IoT معمولاً با هم ترکیب می‌شوند)، ابعاد ۱۰۲۴تایی که تعادل مناسبی بین دقت و حافظه فراهم می‌کند، و قابلیت اجرا روی کارت گرافیک مصرفی، انتخاب شد. این مدل با وزن F16 روی llama.cpp سرو می‌شود که بهینه‌سازی Flash Attention را پشتیبانی می‌کند.

۵.۲ پایگاه‌های داده برداری

پایگاه داده برداری، نوعی از سیستم‌های ذخیره‌سازی است که برای جست‌وجوی شباهت در میان بردارهای ابعاد بالا بهینه شده است. این پایگاه‌ها با استفاده از الگوریتم‌های Approximate Nearest Neighbor (ANN) مانند HNSW، IVF یا PQ، امکان جست‌وجوی میلیون‌ها بردار را در میلی‌ثانیه‌ها فراهم می‌کنند.

FAISS [۱۸] که توسط Facebook AI Research توسعه یافته، یکی از قدیمی‌ترین و پایدارترین کتابخانه‌های جست‌وجوی برداری است. این کتابخانه با پشتیبانی از GPU، انواع مختلف ایندکس از IndexFlatIP تا IVF-PQ، و رابط Python ساده، به استاندارد صنعتی تبدیل شده است. در این پروژه از IndexFlatIP استفاده شده است که با وجود سادگی، برای مجموعه‌های با اندازه متوسط (تا چند میلیون بردار) دقیق و سریع است و امتیاز خروجی آن معادل شباهت کسینوسی است (به شرط نرمال‌سازی ورودی‌ها).

Milvus [۱۹] یک پایگاه داده برداری توزیع‌شده است که برای مقیاس بسیار بزرگ (میلیاردها بردار) طراحی شده و معماری ابری دارد. Weaviate [۲۰] و Qdrant [۲۱] نیز گزینه‌های متن‌باز با قابلیت‌های پیشرفته مانند فیلتر متادیتا و hybrid search هستند. در پروژه حاضر، FAISS به دلیل سادگی، عدم نیاز به سرویس جداگانه و کفایت برای حجم اسناد IoT (چند صد هزار چانک)، انتخاب شد.

۶.۲ Tool Calling و Function Calling

Tool Calling یا Function Calling، قابلیتی است که به LLM اجازه می‌دهد در طول تولید پاسخ، توابع خارجی را فراخوانی کند و از خروجی آن‌ها برای ادامه تولید استفاده کند. این قابلیت اولین بار توسط OpenAI [۲۲] به‌صورت رسمی معرفی شد و سپس در مدل‌های متن‌باز مانند Hermes [۲۳] و Qwen پشتیبانی شد.

شیک و همکاران [۲۴] با معرفی Toolformer، نشان دادند که می‌توان به مدل آموزش داد که خود به‌صورت خودکار تشخیص دهد چه زمانی به ابزار نیاز دارد و چگونه آن را فراخوانی کند. این کار با self-supervised learning انجام می‌شود و به مدل اجازه می‌دهد بدون دخالت انسان، مهارت استفاده از ابزارها را بیاموزد.

یاو و همکاران [۲۵] با معرفی چارچوب ReAct، ترکیب استدلال (Reasoning) و عمل (Acting) را در LLMها پیشنهاد دادند. در این رویکرد، مدل به‌صورت متناوب استدلال می‌کند، اقدامی انجام می‌دهد (مانند فراخوانی ابزار)، نتیجه را مشاهده می‌کند و دوباره استدلال می‌کند. این چرخه تا رسیدن به پاسخ نهایی ادامه می‌یابد و دقیقاً همان الگویی است که در پایپلاین این پروژه پیاده‌سازی شده است.

قالب Hermes [۲۳] که توسط Nous Research توسعه یافته، یکی از استانداردهای متن‌باز برای فراخوانی ابزار است. در این قالب، فراخوانی ابزار داخل تگ‌های `<tool_call>` و `</tool_call>` قرار می‌گیرد و داخل آن یک شیء JSON با فیلدهای `name` و `arguments` نوشته می‌شود. این قالب به‌صورت گسترده در مدل‌های Qwen، Mistral و LLaMA پشتیبانی می‌شود و به همین دلیل در این پروژه انتخاب شد.

۷.۲ اکوسیستم پروتکل‌های IoT

دامنه دانش این پروژه، پروتکل‌های ارتباطی IoT هستند که تنوع بسیار گسترده‌ای دارند. در ادامه، مهم‌ترین این پروتکل‌ها و استانداردهای مرتبط مرور می‌شوند:

۱.۷.۲ پروتکل MQTT

MQTT [۲۶] یک پروتکل پیام‌رسانی سبک وزن است که بر پایه الگوی publish/subscribe کار می‌کند و برای ارتباط ماشین به ماشین (M2M) در محیط‌های با پهنای باند محدود طراحی شده است. نسخه ۵.۰ این پروتکل در سال

۲۰۱۹ توسط OASIS به‌عنوان استاندارد منتشر شد و قابلیت‌هایی مانند QoS سه‌سطحی، retained messages، last will و topic aliases را ارائه می‌دهد. این پروتکل به‌طور گسترده در سیستم‌های هوشمندسازی ساختمان، پایش صنعتی و اینترنت اشیا صنعتی (IIoT) استفاده می‌شود.

۲.۷.۲ پروتکل CoAP

CoAP [۲۷] یک پروتکل لایه کاربردی است که توسط IETF برای محیط‌های دارای محدودیت (constrained environments) طراحی شده است. این پروتکل بر پایه UDP و با الگوی REST کار می‌کند و به دلیل سبک بودن، گزینه مناسبی برای حسگرها و actuatorهای با منابع محاسباتی محدود است. نسخه‌های جدیدتر CoAP روی TCP، TLS و WebSockets نیز پیاده‌سازی شده‌اند [۲۸].

۳.۷.۲ LoRaWAN و LPWAN

LoRaWAN [۲۹، ۳۰] یک پروتکل شبکه LPWAN است که با مدولاسیون Chirp Spread Spectrum (CSS) کار می‌کند و قادر است در فواصل چند کیلومتری با مصرف انرژی بسیار پایین ارتباط برقرار کند. این پروتکل در سه کلاس A، B و C با تعادل متفاوت بین مصرف انرژی و تأخیر تعریف شده است. مشخصات رسمی آن توسط LoRa Alliance منتشر می‌شود.

٤.۷.۲ شبکه‌های 6LoWPAN

6LoWPAN [۳۱] استانداردی است که امکان انتقال IPv6 روی شبکه‌های فیزیکی IEEE 802.15.4 [۳۲] را فراهم می‌کند. این استاندارد با فشرده‌سازی هدر IPv6 (که در حالت عادی ۴۰ بایت است)، امکان استفاده از IPv6 را در شبکه‌های حسگر با MTU کم فراهم می‌کند.

۵.۷.۲ پروتکل Matter

Matter [۳۳] که در سال ۲۰۲۲ توسط Connectivity Standards Alliance معرفی شد، تلاشی برای یکپارچه‌سازی اکوسیستم خانه هوشمند است و توسط شرکت‌هایی مانند Apple، Google، Amazon و Samsung پشتیبانی می‌شود. این پروتکل بر پایه IPv6 و Thread کار می‌کند و هدف آن رفع مشکل عدم هم‌ساختی (interoperability) میان دستگاه‌های مختلف هوشمند است.

٦.۷.۲ ملاحظات امنیتی IoT

NIST در نشریات متعدد [۳۴]، ملاحظات امنیتی و حریم خصوصی در دستگاه‌های IoT را بررسی کرده است. این نشریات چارچوبی برای تولیدکنندگان ارائه می‌دهند تا با رعایت آن، دستگاه‌های امن‌تری تولید کنند. RFC 9175 [۳۵] نیز بهترین شیوه‌های امنیتی فعلی برای CoAP را توضیح می‌دهد که شامل استفاده از DTLS، احراز هویت و رمزنگاری است.

۸.۲ چارچوب‌های توسعه RAG و ابزارهای موجود

چندین چارچوب متن‌باز برای توسعه سیستم‌های RAG وجود دارد که هر یک رویکرد متفاوتی دارند:

LangChain [۳۶] یکی از محبوب‌ترین چارچوب‌ها است که با رابط Python، ابزارهای متنوعی برای ساخت پایپ‌لاین‌های RAG ارائه می‌دهد. این چارچوب با پشتیبانی از انواع LLM، سیستم‌های امبدینگ و پایگاه‌های داده برداری، امکان توسعه سریع را فراهم می‌کند؛ اما به دلیل انتزاع بالا، کنترل دقیق روی جزئیات پایپ‌لاین را محدود می‌کند.

LlamaIndex [۳۷] چارچوب دیگری است که با تمرکز ویژه بر ساخت ایندکس و بازیابی، عملکرد بهتری در سناریوهای پیچیده RAG ارائه می‌دهد. این چارچوب ابزارهای متنوعی برای ساخت ایندکس‌های گرافی، درختی و سلسله‌مراتبی دارد.

در پروژه حاضر، به دلیل نیاز به کنترل دقیق روی هر مرحله پایپ‌لاین (به‌ویژه پارسر Tool Calling و فیلتر استریم)، از چارچوب اختصاصی استفاده شده است که با معماری شیءگرا و ماژولار، امکان تغییر و توسعه آسان هر بخش را فراهم می‌کند.

۹.۲ جمع‌بندی

مرور ادبیات نشان می‌دهد که ترکیب مدل‌های زبانی کوچک اما کارآمد، امبدینگ‌های چندزبانه پیشرفته، پایگاه‌های داده برداری بهینه و معماری Tool-Call-First می‌تواند راه‌حلی جامع برای توسعه دستیارهای تخصصی محلی فراهم کند. در حالی که اکثر سیستم‌های تجاری به سرویس‌های ابری وابسته‌اند، رویکرد این پروژه نشان می‌دهد که با طراحی هوشمندانه، می‌توان همان سطح عملکرد را به‌صورت کاملاً محلی و امن نیز به‌دست آورد. در فصل‌های بعدی، جزئیات پیاده‌سازی هر یک از این اجزا به‌طور دقیق تشریح می‌شود.

فصل ۳

معماری کلی سیستم

۱.۳ مقدمه

در این فصل، معماری کلی سیستم به‌صورت یکپارچه و لایه‌ای ارائه می‌شود. هدف از این طراحی، ایجاد سیستمی است که تمام پردازش‌ها را به‌صورت محلی و مستقل از سرویس‌های ابری انجام دهد، در عین حال قابلیت توسعه، نگهداری و خطایابی آسان را داشته باشد. معماری سیستم بر اساس سه اصل اصلی طراحی شده است: جداسازی مسئولیت‌ها (Separation of Concerns)، ماژولار بودن، و طراحی شیء‌گرا. هر یک از این اصول در طول فصل به‌تفصیل بررسی می‌شوند.

۲.۳ نمای کلی معماری

سیستم از سه لایه اصلی تشکیل شده است که هر یک مسئولیت مشخصی دارد و از طریق رابط‌های مستقل با یکدیگر ارتباط برقرار می‌کنند:

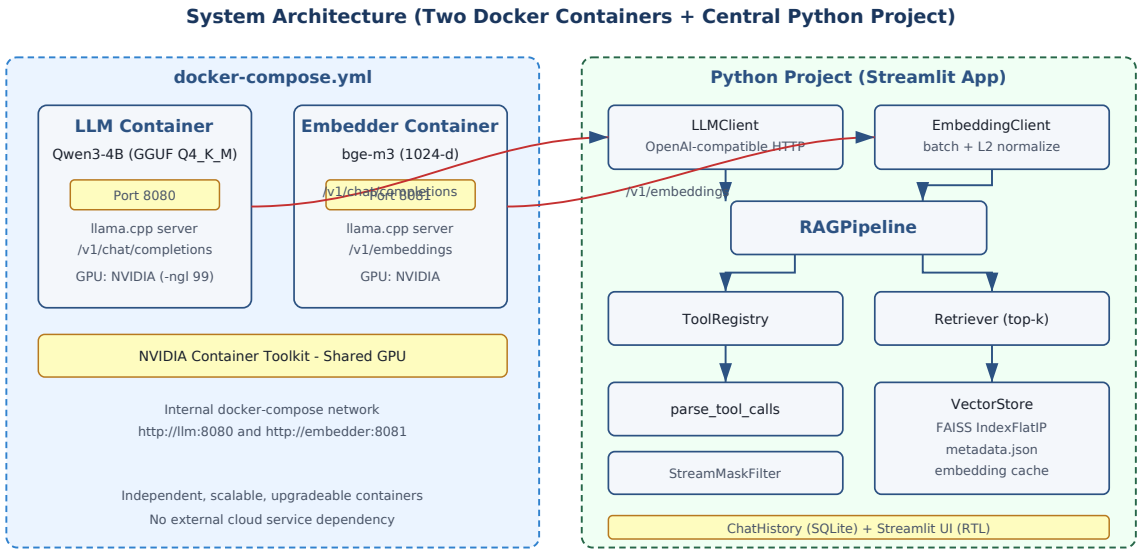
۱. **لایه جمع‌آوری داده (Crawler Layer):** این لایه مسئول کشف، دانلود و ذخیره فایل‌های PDF مرتبط با پروتکل‌های IoT از منابع رسمی است. این لایه به‌صورت یک ماژول مستقل با پیکربندی مجزا عمل می‌کند و خروجی آن، یک مجموعه فایل PDF در یک پوشه تخت است.

۲. **لایه پردازش دانش (Knowledge Processing Layer):** این لایه فایل‌های PDF را به دیتاست CSV تبدیل می‌کند، آن را چانک می‌کند، با bge-m3 امبد می‌کند و در FAISS ذخیره می‌نماید. خروجی این لایه، یک ایندکس آماده برای جست‌وجو است.

۳. **لایه پاسخ‌دهی (Inference Layer):** این لایه شامل پایپلاین RAG، رابط کاربری Streamlit و پایگاه داده تاریخچه SQLite است. این لایه از طریق API سازگار با OpenAI با دو سرویس llama.cpp (یکی برای LLM و دیگری برای امبدینگ) ارتباط برقرار می‌کند.

نمودار زیر (شکل ۱.۳) ساختار لایه‌ای سیستم را به‌صورت بصری نشان می‌دهد. در این نمودار، دو کانتینر Docker در سمت چپ قرار دارند که به ترتیب مدل Qwen3-4B (پورت ۸۰۸۰) و مدل bge-m3 (پورت

۸۰۸۱) را سرو می‌کنند. در سمت راست، پروژه پایتون قرار دارد که شامل اجزای مختلفی مانند LLMClient، EmbeddingClient، RAGPipeline، ToolRegistry، VectorStore و ChatHistory است.



شکل ۱.۳: نمای کلی معماری سیستم با دو کانتینر Docker و پروژه پایتون مرکزی.

۳.۳ انتخاب مدل‌ها و فناوری‌ها

انتخاب هر یک از فناوری‌های استفاده‌شده در این پروژه بر اساس معیارهای مشخصی انجام شده است. در ادامه، انتخاب هر فناوری و دلایل آن شرح داده می‌شود.

۱.۳.۳ انتخاب Qwen3-4B به‌عنوان LLM

مدل Qwen3-4B [۵] از خانواده Qwen3 شرکت Alibaba، با ۴ میلیارد پارامتر انتخاب شد. دلایل این انتخاب عبارتند از:

- **تعادل بین کیفیت و کارایی:** نسخه ۴ میلیارد پارامتر با کوانتایز Q4_K_M تنها حدود ۲٫۵ GB حافظه اشغال می‌کند و روی کارت گرافیک مصرفی با ۸ GB حافظه قابل اجراست.
- **پشتیبانی چندزبانه:** Qwen3 با داده‌های ۱۱۹ زبان از جمله فارسی آموزش دیده است و کیفیت پاسخ‌های فارسی آن قابل قبول است.
- **پشتیبانی از Tool Calling:** این مدل با قالب Hermes سازگار است و می‌تواند فراخوانی ابزار را به‌صورت ساختاریافته تولید کند.
- **مجوز متن‌باز:** Qwen3 با مجوز Apache 2.0 منتشر شده است که استفاده تجاری را نیز مجاز می‌کند.

مقایسه Qwen3-4B با گزینه‌های جایگزین در جدول ۱.۳ ارائه شده است. همان‌طور که مشاهده می‌شود، Gemma-2-9B کیفیت بالاتری دارد اما به‌دلیل حافظه بیشتر و سرعت پایین‌تر، برای استقرار محلی مناسب نیست. LLaMA-3-8B نیز مشکل مشابهی دارد و پشتیبانی فارسی ضعیف‌تری نسبت به Qwen ارائه می‌کند.

جدول ۱.۳: مقایسه مدل‌های زبانی کاندید برای استقرار محلی.

مدل	پارامتر	حافظه با Q4	پشتیبانی فارسی	Tool Calling
Qwen3-4B	4B	2.5 GB	متوسط–خوب	بله (Hermes)
Gemma-2-9B	9B	6 GB	ضعیف	بله
LLaMA-3-8B	8B	5 GB	ضعیف	بله (با Jinja)
Mistral-7B	7B	4.5 GB	ضعیف	بله

۲.۳.۳ انتخاب bge-m3 به‌عنوان امبدینگ

مدل bge-m3 [۱۷] به‌عنوان سرویس امبدینگ انتخاب شد. این مدل با ۵۶۸ میلیون پارامتر، ابعاد ۱۰۲۴تایی، و پشتیبانی از ۱۰۰+ زبان، بهترین گزینه برای اسناد چندزبانه فارسی–انگلیسی است. وزن F16 آن حدود ۱.۲ GB است و روی کارت گرافیک مصرفی به‌راحتی اجرا می‌شود.

۳.۳.۳ انتخاب FAISS به‌عنوان ذخیره‌گاه برداری

FAISS [۱۸] به‌دلیل سادگی، سرعت و عدم نیاز به سرویس جداگانه، انتخاب شد. در حالی که Milvus [۱۹] برای مقیاس میلیاردی مناسب است، برای حجم چند صد هزار چانک، FAISS کفایت می‌کند و با ادغام مستقیم در کد پایتون، نیاز به زیرساخت ابری را از بین می‌برد.

۴.۳ استراتژی Tool-Call-First

نوآوری کلیدی معماری این پروژه، استراتژی «ابزار–محور» [1] است. در RAG کلاسیک، فرآیند به این صورت است:

۱. پرسش کاربر امبد می‌شود.

۲. جست‌وجوی معنایی در FAISS انجام می‌شود.

۳. top-k نتایج در پرامپت مدل قرار می‌گیرند.

۴. مدل پاسخ تولید می‌کند.

[1] Tool-Call-First

این رویکرد یک مشکل اساسی دارد: برای هر سوال، حتی برای سوالات عمومی مانند «سلام» یا «ممنون»، یک جست‌وجوی معنایی انجام می‌شود که نه تنها بی‌فایده است، بلکه بافت غیرمرتبط به مدل می‌فرستد و پاسخ را تحت تأثیر قرار می‌دهد.

در استراتژی Tool-Call-First، فرآیند به این صورت تغییر می‌کند:

۱. پیام کاربر **مستقیماً** به مدل ارسال می‌شود (بدون هیچ امبدینگ یا جست‌جویی).

۲. مدل خود تصمیم می‌گیرد که آیا به دانش‌نامه نیاز دارد یا خیر.

۳. اگر نیاز داشت، یک بلوک `<tool_call>` در خروجی خود تولید می‌کند.

۴. پارسر سیستم این بلوک را تشخیص می‌دهد، ابزار را اجرا می‌کند و نتیجه را به مدل برمی‌گرداند.

۵. مدل با توجه به نتیجه ابزار، پاسخ نهایی را تولید می‌کند.

این استراتژی دو مزیت کلیدی دارد: نخست، برای سوالات عمومی هیچ منابع محاسباتی هدر نمی‌رود؛ و دوم، مدل می‌تواند تصمیم بگیرد چه عبارت جست‌وجویی بهتر است — که معمولاً بهتر از جست‌وجوی مستقیم سوال کاربر عمل می‌کند، چرا که مدل می‌تواند کلمات کلیدی را تشخیص دهد.

۵.۳ سناریوی کامل از پرسش تا پاسخ

برای درک بهتر جریان کامل سیستم، سناریوی زیر را در نظر بگیرید: کاربر در رابط Streamlit می‌پرسد «حداکثر اندازه payload در MQTT 5.0 چقدر است؟». جریان پاسخ‌دهی به‌صورت گام‌به‌گام در ادامه توضیح داده می‌شود:

۱. رابط Streamlit پیام کاربر را دریافت می‌کند و در SQLite ذخیره می‌نماید.

۲. تاریخچه گفتگو (با عمق حافظه قابل تنظیم) به قالب OpenAI تبدیل می‌شود.

۳. پیام کاربر به‌علاوه تاریخچه، به‌علاوه پرامپت سیستمی (شامل آموزش Tool Calling) به کلاس RAGPipeline ارسال می‌شود.

۴. RAGPipeline از طریق LLMClient یک درخواست `POST /v1/chat/completions` با `stream=true` به سرویس Qwen3 روی پورت ۸۰۸۰ می‌فرستد.

۵. مدل شروع به تولید توکن‌ها می‌کند. در ابتدای خروجی، یک بلوک `<tool_call>` قرار می‌گیرد که شامل فراخوانی `search_knowledge_base` با آرگومان `query="MQTT 5.0 payload size"` و `top_k=5` است.

۶. در حین تولید، StreamMaskFilter توکن‌ها را به‌صورت زنده فیلتر می‌کند و بلوک `<tool_call>` را از نمایش کاربر پنهان می‌نماید.

۷. `parse_tool_calls` بلوک `<tool_call>` را از متن خام استخراج می‌کند و یک شیء ToolCall می‌سازد.

۸. ToolRegistry.execute ابزار را با آرگومان‌های مشخص‌شده اجرا می‌کند: ابتدا Retriever.search عبارت جست‌وجو را با `EmbeddingService.embed_query` به بردار تبدیل می‌کند، سپس VectorStore.search در FAISS جست‌وجو می‌کند و top-5 چانک‌های مرتبط (شامل OASIS MQTT v5.0) را برمی‌گرداند.

۹. نتیجه ابزار (شامل متن چانک‌ها، نام منبع و امتیاز شباهت) به قالب JSON تبدیل و در بلوک `<tool_response>` به مدل برمی‌گردد.

۱۰. مدل با مشاهده نتیجه، پاسخ نهایی را به زبان فارسی تولید می‌کند: «حداکثر اندازه payload در MQTT 5.0 به ۲۵۶ مگابایت می‌رسد...».

۱۱. پاسخ نهایی در SQLite ذخیره و منابع استفاده‌شده در سایدبار Streamlit نمایش داده می‌شوند.

کل این فرآیند، در حدود ۵ تا ۸ ثانیه طول می‌کشد که شامل ۲ ثانیه استنتاج مدل (روی کارت گرافیک) و ۳ تا ۵ ثانیه برای امبدینگ پرس‌وجو و جست‌وجوی FAISS است.

۶.۳ جمع‌بندی

در این فصل، معماری کلی سیستم به‌صورت لایه‌ای و ماژولار معرفی شد. سه لایه اصلی (جمع‌آوری داده، پردازش دانش و پاسخ‌دهی) با مسئولیت‌های مشخص جدا شده‌اند و از طریق رابط‌های مستقل ارتباط برقرار می‌کنند. استراتژی Tool-Call-First به‌عنوان نوآوری کلیدی، تصمیم جست‌وجو را به خود مدل می‌سپارد و کارایی سیستم را بهبود می‌بخشد. در فصل‌های بعدی، هر یک از این اجزا به‌طور دقیق و با نمونه کد تشریح می‌شوند.

فصل ٤

جمع‌آوری داده‌ها (Crawler)

۱.٤ مقدمه

اولین گام در ساخت هر سیستم RAG، جمع‌آوری اسناد مرتبط و معتبر است. در پروژه حاضر، هدف جمع‌آوری فایل‌های PDF مرتبط با پروتکل‌های IoT از منابع رسمی و قابل اعتماد است. این فصل، ماژول Crawler پروژه را به‌تفصیل شرح می‌دهد؛ از انتخاب منابع گرفته تا الگوریتم‌های کشف، دانلود و اعتبارسنجی.

طراحی خزنده بر اساس چند اصل اساسی است: اول، **تنها منابع رسمی و رایگان** استفاده می‌شوند تا حقوق مالکیت معنوی رعایت شود. دوم، خزنده باید **مودبانه (polite)** باشد — یعنی درخواست‌های متوالی به یک منبع را با وقفه مناسب انجام دهد تا به سرویس‌دهنده فشار نیاورد. سوم، خزنده باید **مقاوم در برابر خطا** باشد — اگر یک منبع مسدود شود یا قطع شود، سایر منابع باید بدون وقفه ادامه دهند. چهارم، خزنده باید **قابلیت ادامه (resume)** داشته باشد — اگر اجرا قطع و دوباره اجرا شود، فایل‌های قبلی از نو دانلود نشوند.

۲.٤ منابع پشتیبانی‌شده

خزنده از هشت منبع رسمی به‌صورت هم‌زمان پشتیبانی می‌کند. هر منبع با کلاس اختصاصی خود پیاده‌سازی شده است که از کلاس پایه BaseSource ارث‌بری می‌کند. این طراحی که از الگوی Strategy پیروی می‌کند، اجازه می‌دهد بدون تغییر در کد اصلی، منابع جدید به‌راحتی اضافه شوند. جدول ۱.۴ فهرست کامل منابع، نوع محتوا و روش دسترسی را نشان می‌دهد.

جدول ۱.۴: منابع پشتیبانی‌شده توسط خزنده و روش دسترسی به آن‌ها.

منبع	نوع محتوا	روش دسترسی
arxiv	پیش‌چاپ‌های علمی (Access Open)	API رسمی export.arxiv.org
rfc_editor	استانداردهای رسمی IETF (RFC)	پارس rfc-index.txt + PDF رسمی
nist	نشریات NIST (امنیت IoT)	صفحات CSRC + PDFهای nvlpubs
oasis	استانداردهای MQTT 5.0، MQTT 3.1.1، AMQP، MQTT-SN، Sparkplug	لینک مستقیم رسمی
lora_alliance	مشخصات رسمی LoRaWAN L2 1.0.4 و RP002	توکن دانلود رسمی هاب LoRa Alliance
semantic_scholar	مقالات Open Access (فقط هاست‌های معتبر)	Graph API رایگان
mdpi	مقالات ژورنال‌های Sensors، Electronics، IoT	DOAJ + OpenAlex
ietf_datatracker	اینترنت‌درافت‌های گروه‌های کاری IoT	REST API رسمی Datatracker

۳.٤ الگوی طراحی Strategy و کلاس پایه

برای پشتیبانی از منابع متعدد با ساختارهای متفاوت، از الگوی طراحی Strategy استفاده شده است. در این الگو، یک کلاس پایه به‌نام BaseSource رابط مشترکی را تعریف می‌کند که همه منابع باید آن را پیاده‌سازی کنند. این رابط شامل متد اصلی discover() است که لیستی از اشیاء Document (شامل URL، عنوان، سال و شناسه) را برمی‌گرداند.

کلاس پایه همچنین چندین قابلیت مشترک را فراهم می‌کند که هر منبع می‌تواند از آن‌ها بهره ببرد: deduplication (با استفاده از مجموعه seen_urls جهانی)، politeness (با وقفه قابل تنظیم برای هر منبع)، error isolation (اگر یک منبع خطا دهد، سایر منابع ادامه می‌دهند) و progress bar (با tqdm).

```python
class BaseSource:
    """
    name = "base"
    label = "src"
    display = "Base"

    def __init__(self, session, limit: int, min_year: int, verbose: bool
        = True):
        self.session = session
        self.limit = limit
```

```
        self.min_year = min_year
        self.verbose = verbose
        self.delay = self._delay_for()

    def discover(self) -> List[Document]:
        raise NotImplementedError

    def collect(self, seen_urls: set) -> List[Document]:
        """"""
        docs: List[Document] = []
        try:
            raw = self.discover()
        except Exception as exc:
            utils.say(f"!! source '{self.name}' failed: {exc}")
            raw = []
        unique = []
        for doc in raw:
            key = doc.url.rstrip("/")
            if key in seen_urls:
                continue
            seen_urls.add(key)
            unique.append(doc)
        return unique[: self.limit]
```

٤.٤ کلمات کلیدی و فیلتر موضوعی

برای اینکه خزنده فقط فایل‌های مرتبط با IoT را جمع‌آوری کند، از یک لیست گسترده کلیدواژه‌ها استفاده می‌شود که در فایل `config.py` تعریف شده است. این کلیدواژه‌ها شامل نام پروتکل‌ها، استانداردها و مفاهیم مرتبط با IoT هستند:

```
IOT_KEYWORDS = [
    "MQTT", "CoAP", "AMQP", "6LoWPAN", "RPL", "LoRaWAN", "LoRa",
    "NB-IoT", "Sigfox", "Zigbee", "Thread", "Matter", "BLE",
    "Bluetooth Low Energy", "LPWAN", "SCHC", "wireless sensor network",
    "IoT security", "IIoT", "industrial IoT", "IoT protocol",
    "802.15.4", "OPC UA", "XMPP", "WebSocket", "M2M", "IoT
```

```
    interoperability",
]
```

این کلیدواژه‌ها به دو دسته تقسیم می‌شوند: کلیدواژه‌هایی که به‌تنهایی خاص IoT هستند (مانند MQTT، CoAP، LoRaWAN) که در arXiv به‌صورت مستقل جست‌جو می‌شوند؛ و کلیدواژه‌های عمومی‌تر که با ترکیب با IoT جست‌جو می‌شوند تا نتایج نامرتبط فیلتر شوند. این منطق در متد `_build_query` از کلاس ArxivSource پیاده‌سازی شده است.

۵.٤ فیلتر سال انتشار

برای جلوگیری از جمع‌آوری مقالات بسیار قدیمی، یک فیلتر سال انتشار اعمال می‌شود که به‌صورت پیش‌فرض، مقالات قبل از سال ۲۰۲۰ را حذف می‌کند. این مقدار از طریق آرگومان min-year– قابل تنظیم است. با این حال، استانداردهای مرجع مانند OASIS، LoRa و NIST از این فیلتر مستثنی هستند، زیرا این اسناد «مرجعِ» پروتکل هستند و تعدادشان محدود است؛ حتی اگر سال انتشار آن‌ها قدیمی باشد، همچنان معتبر و ضروری هستند.

٦.٤ اعتبارسنجی PDF

یکی از چالش‌های رایج در خزنده‌های وب، تشخیص فایل‌های PDF واقعی از صفحات خطای HTML یا صفحات ورود به سیستم است. برای حل این مشکل، خزنده از یک اعتبارسنجی دو-مرحله‌ای استفاده می‌کند:

۱. **بررسی magic bytes:** فایل‌های PDF واقعی همیشه با ۵ بایت -PDF% شروع می‌شوند. خزنده در ابتدای دانلود، اولین chunk را بررسی می‌کند و اگر این امضا وجود نداشت، دانلود را متوقف و فایل را دور می‌ریزد.

۲. **بررسی حداقل حجم:** فایل‌های PDF معتبر معمولاً حداقل ۲۰ کیلوبایت هستند. اگر حجم فایل دانلودشده کمتر از این مقدار باشد، احتمالاً یک صفحه خطا یا یک PDF ناقص است و فایل حذف می‌شود.

این رویکرد باعث می‌شود خزنده به‌صورت خودکار صفحات خطای HTML، چالش‌های Cloudflare و صفحات login که به‌جای PDF برمی‌گردند را فیلتر کند، بدون اینکه کاربر نیاز به دخالت داشته باشد.

۷.٤ قابلیت Resume

برای پشتیبانی از قطع و وصل اجرا، خزنده از یک مکانیزم resume استفاده می‌کند. نام هر فایل به‌صورت قطعی بر اساس SHA-1 آدرس URL ساخته می‌شود:

```
def target_filename(doc, used_names: set) -> str:
```

```
    """ : <source>_<slug>_<hash8>.pdf"""
    slug = sanitize_title(doc.title) or "document"
    base = f"{doc.source}_{slug}_{sha1_8(doc.url)}.pdf"
    name, counter = base, 1
    while name in used_names:
    name = base.replace(".pdf", f"_{counter}.pdf")
    counter += 1
    used_names.add(name)
    return name
```

هنگام اجرای مجدد خزنده، اگر فایلی با این نام از قبل وجود داشته باشد و یک PDF معتبر باشد (با امضای %PDF- و حجم کافی)، خزنده آن را **نادیده می‌گیرد** و به سراغ فایل بعدی می‌رود. این کار باعث می‌شود خزنده در صورت قطع برق، خطای شبکه یا توقف دستی، از همان‌جا ادامه یابد و هیچ فایل مجدداً دانلود نشود.

۸.٤ نمونه: کلاس ArxivSource

برای درک بهتر نحوه پیاده‌سازی یک منبع، کلاس ArxivSource را در نظر بگیرید که از API رسمی arXiv استفاده می‌کند. این کلاس از Atom XML API در آدرس `export.arxiv.org/api/query` استفاده می‌کند که به‌صورت رایگان و بدون نیاز به کلید API در دسترس است.

```
class ArxivSource(BaseSource):
    name = "arxiv"
    label = "arxiv"
    display = "arXiv (open-access preprints)"

    def discover(self):
    docs = []
    queries = self.QUERIES[:10]
    with self.query_progress(queries, "arXiv queries"):
    for keyword in queries:
    url = (
    f"{ARXIV_API}?search_query={quote(self._build_query(keyword))}"
    f"&start=0&max_results={cfg.ARXIV_MAX_RESULTS_PER_QUERY}"
    "&sortBy=submittedDate&sortOrder=descending"
    )
    try:
    resp = self.session.get(url, timeout=cfg.REQUEST_TIMEOUT)
```

```
        if resp.ok:
            docs.extend(self._parse_atom(resp.text))
    except Exception:
        pass
    self.sleep()
    return self._year_filter(docs)
```

نکته مهم در این کد، رعایت درخواست رسمی arXiv برای حداقل ۳ ثانیه تأخیر بین درخواست‌هاست که در متد self.sleep() انجام می‌شود. همچنین، هر خطا در یک درخواست (مثلاً قطع موقت شبکه) با بلاک `try/except` مهار می‌شود و باعث توقف کل خزنده نمی‌شود.

۹.٤ Politeness و مدیریت نرخ

هر منبع، یک تأخیر متفاوت برای درخواست‌های متوالی دارد که در دیکشنری `SOURCE_DELAYS` تعریف شده است:

```
SOURCE_DELAYS = {
    "arxiv": 3.0,
    "rfc_editor": 1.0,
    "ietf_datatracker": 2.0,
    "nist": 1.0,
    "mdpi": 2.0,
    "semantic_scholar": 6.0,
    "oasis": 1.0,
    "lora_alliance": 1.0,
}
```

اگر کاربر در شبکه‌ای با محدودیت نرخ (rate limiting) مواجه شد، می‌تواند با آرگومان `--delay-factor 2.0` تمام تأخیرها را دو برابر کند تا خزنده ملایم‌تر عمل کند. علاوه بر این، خزنده از retry با backoff نمایی برای خطاهای 429 (Too Many Requests) و 5xx استفاده می‌کند که در کلاس `utils.build_session` پیاده‌سازی شده است.

۱۰.٤ عایق‌سازی خطا

یکی از مهم‌ترین ویژگی‌های خزنده، **عایق‌سازی خطا** است. اگر یک منبع (مثلاً Cloudflare روی MDPI) مسدود شود، بقیه منابع باید بدون وقفه ادامه دهند. این کار با بلاک `try/except` در متد `collect()` از کلاس پایه انجام می‌شود:

- اگر `discover()` خطا بدهد، `collect()` یک لیست خالی برمی‌گرداند و به سراغ منبع بعدی می‌رود.
- در پایان اجرا، یک گزارش آماری نمایش داده می‌شود که برای هر منبع، تعداد فایل‌های دانلودشده، شکست‌خورده، صرف‌نظرشده (موجود از قبل) و نامعتبر را نشان می‌دهد.

۱۱.٤ خلاصه اجرای خزنده

اجرای کامل خزنده با تنظیمات پیش‌فرض (۶۰ فایل از هر منبع) حدود ۳۰ تا ۴۵ دقیقه طول می‌کشد و حدود ۴۸۰ فایل PDF (با حجم تقریبی ۲ تا ۴ GB) تولید می‌کند. خروجی در یک پوشه تخت به‌نام `pdfs/` ذخیره می‌شود که نام هر فایل با پیشوند نام منبع شروع می‌شود (مانند `arxiv_mqtt_security_1a2b3c4d.pdf`). این فایل‌ها در مرحله بعد، به دیتاست CSV تبدیل می‌شوند که در فصل پنجم به‌تفصیل شرح داده می‌شود.

فصل ۵

تبدیل به دیتاست و نرمال‌سازی

۱.۵ مقدمه

پس از جمع‌آوری فایل‌های PDF، مرحله بعد تبدیل آن‌ها به یک دیتاست ساختاریافته است که برای چانکینگ و امبدینگ قابل استفاده باشد. در این پروژه، فرمت انتخاب‌شده CSV است؛ زیرا ساده، سازگار با تمامی ابزارهای تحلیل داده و قابل بازبینی با ویرایشگرهای متنی یا Excel است. این فصل، فرمت دیتاست، نحوه استخراج متن از PDF و فرآیند نرمال‌سازی را شرح می‌دهد.

۲.۵ فرمت دیتاست CSV

دیتاست یک فایل CSV با دو ستون اصلی است که در سطر اول، نام ستون‌ها به‌عنوان هدر قرار می‌گیرد:

جدول ۱.۵: فرمت دیتاست CSV.

نام ستون	توضیح
filename	نام فایل مبدأ؛ مثلاً `report.pdf`. نام‌های قابل قبول دیگر: `file_name`، `file`، `name`، `source`، `title`
content	متن کامل استخراج‌شده از فایل. نام‌های قابل قبول دیگر: `text`، `body`، `document`، `contents`

قواعد مهم برای فرمت دیتاست:

۱. انکودینگ باید UTF-8 باشد (BOM نیز تحمل می‌شود).

۲. سطر اول هدر است و نام ستون‌ها در آن می‌آید.

۳. پاراگراف‌ها داخل ستون content با یک خط خالی (\n\n) از هم جدا می‌شوند — چانکر دقیقاً روی همین مرزها عمل می‌کند و چیزی از بین نمی‌رود.

۴. سلول‌های چندخطی طبق استاندارد CSV داخل کوتیشن دوتایی (" ") قرار می‌گیرند.

۵. هر ردیف معادل یک فایل PDF (یا هر سند دیگر) است.

۳.۵ استخراج متن از PDF

برای استخراج متن از فایل‌های PDF، از کتابخانه pypdf در پایتون استفاده می‌شود. این کتابخانه با پشتیبانی از اکثر فرمت‌های PDF متنی، قادر است متن هر صفحه را به‌صورت جداگانه استخراج کند. نمونه کد تولید دیتاست از یک پوشه PDF در ادامه ارائه شده است:

```
import csv
from pathlib import Path
from pypdf import PdfReader

with open("data/data.csv", "w", encoding="utf-8", newline="") as out:
 writer = csv.writer(out)
 writer.writerow(["filename", "content"])
 for pdf_path in sorted(Path("pdfs").glob("*.pdf")):
 reader = PdfReader(pdf_path)
 pages = [(page.extract_text() or "").strip() for page in
   reader.pages]

 content = "\n\n".join(page for page in pages if page)
 writer.writerow([pdf_path.name, content])
print(" : data/data.csv")
```

۴.۵ نرمال‌سازی متن

متن استخراج‌شده از PDF معمولاً دارای مشکلاتی است که باید قبل از چانکینگ رفع شوند. مهم‌ترین این مشکلات عبارتند از:

- **بایت‌های NUL:** هنگام استخراج متن از PDFهای قدیمی یا خراب، ممکن است بایت‌های NUL (\x00) در متن قرار گیرند که باعث خطای `NUL contains line` در پارسر CSV می‌شوند. چانکر این بایت‌ها را قبل از پردازش حذف می‌کند.
- **انکودینگ:** فایل CSV ممکن است با UTF-8 BOM ذخیره شود (خروجی پیش‌فرض Excel). چانکر از `utf-8-sig` برای خواندن استفاده می‌کند که BOM را به‌صورت خودکار تشخیص و حذف می‌کند.

- **فاصله‌های اضافی:** متن استخراج‌شده ممکن است دارای فاصله‌های متعدد، خطوط خالی متوالی یا تب‌های زائد باشد که در مرحله چانکینگ با regex نرمال می‌شوند.
- **کاراکترهای کنترلی:** کاراکترهای غیرچاپی مانند BEL، BS، VT که در استخراج برخی PDFها ظاهر می‌شوند، فیلتر می‌گردند.

کلاس CSVLoader که در ماژول `chunker.py` پیاده‌سازی شده، تمامی این موارد را به‌صورت خودکار مدیریت می‌کند و لیستی از اشیاء Document (شامل `source` و `content`) را برمی‌گرداند.

۵.۵ پشتیبانی از ستون‌های با نام‌های مختلف

برای انعطاف‌پذیری بیشتر، چانکر از نام‌های مختلفی برای ستون‌ها پشتیبانی می‌کند. اگر کاربر دیتاستی با نام ستون متفاوت از filename و content داشته باشد، چانکر به‌صورت خودکار نام ستون صحیح را با تطبیق case-insensitive پیدا می‌کند:

- نام ستون «نام فایل»: `filename`، `file_name`، `file`، `name`، `source`، `title`
- نام ستون «محتوا»: `content`، `text`، `body`، `document`، `contents`

اگر هیچ‌یک از نام‌های قابل قبول یافت نشد، چانکر یک خطای واضح با نام ستون‌های موجود برمی‌گرداند تا کاربر بتواند دیتاست را اصلاح کند.

۶.۵ آمار دیتاست نهایی

پس از اجرای خزنده و ساخت دیتاست، آمار نمونه به شرح زیر است:

- تعداد فایل‌های PDF جمع‌آوری‌شده: حدود ۴۸۰ فایل از ۸ منبع
- حجم کل: حدود ۱۵ GB
- تعداد کاراکتر کل متن استخراج‌شده: حدود ۱۵۰ میلیون کاراکتر
- میانگین طول هر سند: حدود ۳۰۰ هزار کاراکتر
- زبان غالب: انگلیسی (با برخی مقالات فارسی در صورت وجود)

این دیتاست در فایل `data/data.csv` ذخیره می‌شود و در فصل بعد، توسط چانکر پاراگراف‌محور به تکه‌های کوچک‌تر تبدیل می‌شود.

فصل ٦

تکه‌تکه‌سازی (Chunking)

۱.٦ مقدمه

چانکینگ (Chunking)، فرآیند تقسیم متن بلند یک سند به تکه‌های کوچک‌تر (chunks) است که برای امبدینگ و بازیابی بهینه باشند. این مرحله یکی از حیاتی‌ترین بخش‌های هر سیستم RAG است، زیرا کیفیت چانک‌ها مستقیماً بر کیفیت پاسخ‌ها تأثیر می‌گذارد: اگر چانک‌ها خیلی بزرگ باشند، امبدینگ نمی‌تواند معنای دقیق را ثبت کند و اگر خیلی کوچک باشند، بافت کافی برای پاسخ‌دهی فراهم نمی‌شود.

در این پروژه، یک چانکر پاراگراف‌محور با تضمین **پوشش کامل متن** پیاده‌سازی شده است؛ یعنی هیچ پاراگرافی از چانک‌ها جا نمی‌افتد و هیچ متنی از بین نمی‌رود. این تضمین با یک راستی‌آزمایی خودکار پس از چانکینگ اعتبارسنجی می‌شود که در صورت بروز اشکال، خطا گزارش می‌کند.

۲.٦ الگوریتم چانکینگ پاراگراف‌محور

الگوریتم چانکینگ به سه مرحله تقسیم می‌شود:

۱. **مرحله ۱ — شکستن به پاراگراف:** متن با خط خالی (\n\n) به پاراگراف‌ها تقسیم می‌شود. هر پاراگراف که طول آن از حد مجاز (PARAGRAPH_MAX_CHARS = 1500 کاراکتر) تجاوز کند، به‌صورت جمله‌به‌جمله و بدون افت متن به قطعات کوچک‌تر شکسته می‌شود.

۲. **مرحله ۲ — پنجره لغزان روی پاراگراف‌ها:** با یک پنجره لغزان از اندازه CHUNK_PARAGRAPHS = 4 پاراگراف و هم‌پوشانی CHUNK_OVERLAP_PARAGRAPHS = 1 پاراگراف، چانک‌ها ساخته می‌شوند. این یعنی هر چانک شامل ۴ پاراگراف است و چانک بعدی از پاراگراف سوم چانک قبلی شروع می‌شود (با یک پاراگراف هم‌پوشانی).

۳. **مرحله ۳ — راستی‌آزمایی پوشش:** پس از ساخت چانک‌ها، یک راستی‌آزمایی خودکار انجام می‌شود: مجموعه شماره پاراگراف‌های پوشش‌داده‌شده با مجموعه شماره پاراگراف‌های اصلی مقایسه می‌شود و در صورت تفاوت، یک خطا گزارش می‌گردد. همچنین، محتوای چانک‌ها (بدون فاصله‌ها) با محتوای اصلی مقایسه می‌شود تا هیچ متنی از بین نرفته باشد.

۳.۶ سقف کاراکتر هر چانک

علاوه بر تعداد پاراگراف، یک سقف کاراکتر نیز برای هر چانک اعمال می‌شود (`CHUNK_MAX_CHARS = 3000` کاراکتر). این سقف دو دلیل دارد: نخست، جلوگیری از چانک‌های بسیار بزرگ که از حداکثر طول ورودی bge-m3 (۸۱۹۲ توکن) تجاوز کنند؛ و دوم، اطمینان از اینکه هر چانک در پرامپت مدل به‌خوبی جا می‌گیرد. اگر مجموع پاراگراف‌های یک پنجره از این سقف فراتر رود، چانکر پنجره را زودتر می‌بندد و به سراغ پاراگراف بعدی می‌رود.

۴.۶ نمونه کد چانکر

در ادامه، کد اصلی متد `chunk()` از کلاس ParagraphChunker ارائه شده است:

```
def chunk(self, document: Document) -> list[Chunk]:
 """ """
 pieces = self.split_paragraphs(document.content)
 if not pieces:
 return []
 chunks: list[Chunk] = []
 index = 0
 total = len(pieces)
 while index < total:

 window: list[int] = []
 char_count = 0
 for cursor in range(index, min(index + self.paragraphs_per_chunk,
   total)):
 piece = pieces[cursor]
 if window and char_count + len(piece) > self.max_chunk_chars:
 break
 window.append(cursor)
 char_count += len(piece) + 2
 text = "\n\n".join(pieces[i] for i in window)
 chunks.append(Chunk(
 text=text,
 source=document.source,
 chunk_index=len(chunks),
 paragraph_indices=tuple(window),
 char_count=len(text),
 ))
```

```
        index += max(1, len(window) - self.overlap_paragraphs)
    self._verify(document, pieces, chunks)
    return chunks
```

۵.۶ راستی‌آزمایی پوشش کامل

یکی از ویژگی‌های منحصربه‌فرد این چانکر، راستی‌آزمایی خودکار پوشش کامل است. این کار با دو بررسی انجام می‌شود:

۱. **بررسی پوشش پاراگراف‌ها:** مجموعه شماره پاراگراف‌های پوشش‌داده‌شده توسط چانک‌ها (از فیلد `paragraph_indices`) با مجموعه `range(len(pieces))` مقایسه می‌شود. اگر تفاوتی وجود داشته باشد، یعنی برخی پاراگراف‌ها جا مانده‌اند و خطا گزارش می‌شود.

۲. **برابر محتوایی:** متن اصلی (با حذف تمام فاصله‌ها) با مجموع متن پاراگراف‌ها مقایسه می‌شود. اگر تفاوتی وجود داشته باشد، یعنی در فرآیند شکستن پاراگراف‌ها، متنی از بین رفته یا تغییر کرده است.

```
@staticmethod
def _verify(document: Document, pieces: list[str], chunks:
    list[Chunk]) -> None:
 """ """
 covered: set[int] = set()
 for chunk in chunks:
 covered.update(chunk.paragraph_indices)
 if covered != set(range(len(pieces))):
 raise DatasetError(
 f" and «{document.source}»: "
 f"some paragraphs missed by chunks."
 )
 original = _WHITESPACE_RE.sub("", document.content)
 rebuilt = _WHITESPACE_RE.sub("", "".join(pieces))
 if original != rebuilt:
 raise DatasetError(
 f" «{document.source}» ."
 )
```

۶.۶ گزارش آماری چانکینگ

پس از چانکینگ کامل دیتاست، یک گزارش آماری نمایش داده می‌شود که شامل اطلاعات زیر است:

- تعداد اسناد (فایل‌ها)
- تعداد چانک‌های تولیدشده
- مجموع کاراکتر متن اصلی
- مجموع کاراکتر چانک‌ها (با احتساب هم‌پوشانی)
- میانگین طول چانک (کاراکتر)
- بلندترین چانک (کاراکتر)
- فهرست فایل‌ها

این گزارش به کاربر کمک می‌کند تا کیفیت چانکینگ را ارزیابی کند و در صورت نیاز، پارامترها (تعداد پاراگراف، هم‌پوشانی، سقف کاراکتر) را تنظیم نماید.

۷.۶ تنظیمات پیش‌فرض چانکینگ

تنظیمات پیش‌فرض چانکینگ که در فایل env. قابل تغییر هستند:

جدول ۱.۶: تنظیمات پیش‌فرض چانکینگ.

کلید	پیش‌فرض	توضیح
CHUNK_PARAGRAPHS	۴	تعداد پاراگراف در هر چانک
CHUNK_OVERLAP_PARAGRAPHS	۱	هم‌پوشانی پاراگرافی بین چانک‌ها
CHUNK_MAX_CHARS	۳۰۰۰	سقف کاراکتر هر چانک
PARAGRAPH_MAX_CHARS	۱۵۰۰	سقف کاراکتر یک پاراگراف پیش از ریزشگی

این تنظیمات بر اساس تجربه عملی و آزمون‌وخطا به‌دست آمده‌اند و تعادل خوبی بین دقت بازیابی و کارایی امبدینگ فراهم می‌کنند. برای اسناد فنی با ساختار پاراگراف‌های کوتاه (مانند RFCها)، مقادیر پیش‌فرض مناسب هستند؛ اما برای اسناد با پاراگراف‌های بلند (مانند مقالات دانشگاهی)، ممکن است کاهش CHUNK_PARAGRAPHS به ۲ یا ۳ مفید باشد.

۸.۶ جمع‌بندی

چانکینگ پاراگراف‌محور با تضمین پوشش کامل، یکی از اجزای حیاتی سیستم RAG است که کیفیت بازیابی را تضمین می‌کند. با تنظیمات پیش‌فرض، هر سند به چانک‌های چندپاراگرافی با هم‌پوشانی مناسب تقسیم می‌شود و راستی‌آزمایی خودکار، اطمینان از عدم افت متن را فراهم می‌کند. در فصل بعد، نحوه امبدینگ این چانک‌ها با مدل bge-m3 بررسی می‌شود.

فصل ۷

سرویس امبدینگ (bge-m3)

۱.۷ مقدمه

پس از چانکینگ، مرحله بعدی تبدیل هر چانک متنی به یک بردار عددی است که معنای آن را در فضای ریاضی ثبت می‌کند. این بردارها، پایه جست‌وجوی معنایی هستند: برای یافتن چانک‌های مرتبط با یک پرسش، کافی است شباهت بردار پرسش با بردارهای چانک‌ها محاسبه شود و نزدیک‌ترین‌ها بازیابی شوند. در این پروژه، از مدل bge-m3 [۱۷] به‌عنوان سرویس امبدینگ استفاده شده است که یکی از پیشرفته‌ترین مدل‌های چندزبانه امبدینگ متن است.

۲.۷ معرفی bge-m3

مدل bge-m3 توسط Beijing Academy of Artificial Intelligence (BAAI) توسعه یافته است. نام m3 به سه ویژگی «□» آن اشاره دارد:

- **Multi-Linguality:** پشتیبانی از بیش از ۱۰۰ زبان، شامل فارسی، انگلیسی، عربی، چینی و بسیاری از زبان‌های دیگر.
- **Multi-Functionality:** سه تابع شباهت به‌صورت هم‌زمان: شباهت کسینوسی (dense)، جست‌وجوی کلیدواژه‌ای پراکنده (sparse/lexical) و multi-vector retrieval.
- **Multi-Granularity:** پشتیبانی از ورودی‌های با طول متغیر تا ۸۱۹۲ توکن، که اجازه می‌دهد هم جملات کوتاه و هم اسناد بلند امبد شوند.

این مدل بر پایه معماری XLM-RoBERTa با ۵۶۸ میلیون پارامتر ساخته شده است و با تکنیک self-knowledge distillation آموزش دیده است. در معیارهای ارزیابی استاندارد مانند MTEB، bge-m3 در میان بهترین مدل‌های امبدینگ چندزبانه قرار دارد.

۳.۷ چرا bge-m3؟

انتخاب bge-m3 برای این پروژه بر اساس چند دلیل کلیدی بود:

۱. **پشتیبانی از فارسی:** بسیاری از مدل‌های امبدینگ فقط با داده‌های انگلیسی آموزش دیده‌اند و کیفیت امبدینگ فارسی آن‌ها ضعیف است. bge-m3 با پشتیبانی رسمی از ۱۰۰+ زبان، کیفیت خوبی برای فارسی ارائه می‌دهد.

۲. **ابعاد ۱۰۲٤تایی:** این ابعاد تعادل مناسبی بین دقت و حافظه فراهم می‌کند. ابعاد کوچک‌تر (مانند ۳۸۴ در MiniLM) ممکن است کیفیت بازیابی را کاهش دهد، در حالی که ابعاد بزرگ‌تر (مانند ۱۵۳۶ در OpenAI) حافظه بیشتری نیاز دارند.

۳. **قابلیت اجرای محلی:** bge-m3 با وزن F16 حدود ۱.۲ GB است و روی کارت گرافیک مصرفی به‌راحتی اجرا می‌شود. این برخلاف APIهای تجاری است که نیاز به ارسال داده به سرویس ابری دارند.

۴. **پشتیبانی از طول ورودی ۸۱۹۲ توکن:** این ویژگی اجازه می‌دهد چانک‌های بلندتر (تا حدود ۳۰۰۰ کاراکتر) به‌خوبی امبد شوند، بدون نیاز به برش بیشتر.

٤.۷ استقرار bge-m3 روی llama.cpp

مدل bge-m3 به‌صورت یک کانتینر Docker با استفاده از llama.cpp سرو می‌شود. llama.cpp یک پروژه متن‌باز است که امکان اجرای مدل‌های زبانی و امبدینگ را روی CPU و GPU با بهینه‌سازی بالا فراهم می‌کند. پیکربندی کانتینر امبدینگ در فایل docker-compose.yml به شرح زیر است:

```
embedding:
image: llama-cuda13:local
container_name: bge-m3
restart: unless-stopped
ports:
- "8081:8080"
volumes:
- /mnt/d/AI/models:/models:ro
command:
- -m
- /models/bge-m3-F16.gguf
- --host
- 0.0.0.0
- --port
- "8080"
- --embedding
- --pooling
- "cls"
- -ngl
```

```
      - "99"
      - -c
      - "8192"
      - -b
      - "2048"
      - -ub
      - "2048"
      - -np
      - "4"
      - -lv
      - "3"
      - -fa
      - "on"
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
```

پارامترهای کلیدی در این پیکربندی عبارتند از:

- `--embedding`: فعال‌سازی حالت امبدینگ (به جای تولید متن).
- `--pooling cls`: استفاده از خروجی توکن [CLS] به‌عنوان بردار نهایی (بهترین روش برای bge-m3).
- `-ngl 99`: قرار دادن تمام لایه‌های مدل روی GPU برای استنتاج سریع.
- `-c 8192`: اندازه پنجره زمینه ۸۱۹۲ توکن.
- `-b 2048 -ub 2048`: اندازه دسته پردازش ۲۰۴۸ توکن (هم برای پردازش دسته‌ای و هم برای استنتاج).
- `-np 4`: ۴ اسلات موازی برای پشتیبانی از چندین درخواست هم‌زمان.
- `-fa on`: فعال‌سازی Flash Attention برای کاهش مصرف حافظه و افزایش سرعت.

۵.۷ پردازش دسته‌ای و نرمال‌سازی L2

کلاس EmbeddingService در ماژول `embedder.py`، لایه بالادستی EmbeddingClient است که چند ویژگی مهم را فراهم می‌کند:

۱. **پردازش دسته‌ای (batch):** به‌جای ارسال هر متن به‌صورت جداگانه، چند متن در یک درخواست به سرویس امبدینگ ارسال می‌شوند. این کار با تنظیم `EMBED_BATCH_SIZE = 16`، تعداد درخواست‌ها را ۱۶ برابر کاهش می‌دهد و سرعت کل را به‌طور قابل توجهی افزایش می‌دهد.

۲. **نرمال‌سازی L2:** هر بردار به‌گونه‌ای نرمال می‌شود که نرم آن برابر ۱ باشد. این کار باعث می‌شود ضرب داخلی (که FAISS IndexFlatIP محاسبه می‌کند) معادل شباهت کسینوسی باشد، که معیار استاندارد برای سنجش شباهت معنایی است.

۳. **اعتبارسنجی خروجی:** بررسی اینکه تعداد بردارهای دریافتی با تعداد متن‌های ارسالی برابر باشد و تمام مقادیر عددی معتبر (بدون NaN یا Inf) باشند.

```
class EmbeddingService:
 """"""

 def __init__(self, client: EmbeddingClient, batch_size: int = 16):
 self.client = client
 self.batch_size = max(1, batch_size)

 def embed_texts(self, texts: list[str]) -> np.ndarray:
 """"""
 if not texts:
 return np.zeros((0, 1), dtype="float32")
 vectors: list[list[float]] = []
 total = len(texts)
 for start in range(0, total, self.batch_size):
 batch = texts[start:start + self.batch_size]
 vectors.extend(self.client.embed(batch))
 done = min(start + self.batch_size, total)
 logger.info("Embedded %d of %d texts.", done, total)
 matrix = np.asarray(vectors, dtype="float32")
 if matrix.ndim != 2 or matrix.shape[0] != total:
 raise APIResponseError(
 "[Embedding] Processing texts."
 )
 return self._normalize(matrix)

 @staticmethod
 def _normalize(matrix: np.ndarray) -> np.ndarray:
 """ L2 ( -> 1)."""
```

```
    norms = np.linalg.norm(matrix, axis=1, keepdims=True)
    norms[norms == 0] = 1.0
    return (matrix / norms).astype("float32")
```

۶.۷ تلاش مجدد با Backoff نمایی

هنگام امبدینگ حجم زیادی از داده‌ها، ممکن است سرویس به‌طور موقت در دسترس نباشد (مثلاً به دلیل فشار حافظه یا قطع موقت شبکه). برای مقابله با این موارد، یک لایه retry با backoff نمایی پیاده‌سازی شده است:

```
MAX_RETRIES = 4
BACKOFF_BASE = 2

def embed_batch(service, texts, dim, retries):
    """"""
    last_exc = None
    for attempt in range(retries + 1):
        try:
            arr = to_array(service.embed_texts(texts))
            check_vectors(arr, len(texts), dim)
            return arr
        except KeyboardInterrupt:
            raise
        except Exception as exc:
            last_exc = exc
            if attempt < retries:
                wait = BACKOFF_BASE * (2 ** attempt)
                logger.warning(
                    " (%d ): %s - %d/%d %d ",
                    len(texts), exc, attempt + 1, retries, wait
                )
                time.sleep(wait)
    raise last_exc

def embed_robust(service, texts, dim, retries=MAX_RETRIES):
    """"""
    try:
```

```
        return embed_batch(service, texts, dim, retries)
    except KeyboardInterrupt:
        raise
    except Exception:
        if len(texts) == 1:
            raise
        mid = len(texts) // 2
        logger.warning(" and (%d + %d)", mid, len(texts) - mid)
        left = embed_robust(service, texts[:mid], dim, retries=1)
        right = embed_robust(service, texts[mid:], dim, retries=1)
        return np.vstack([left, right])
```

این لایه مقاومت، دو سطح دارد: نخست، retry ساده با تأخیر نمایی؛ و دوم، اگر حتی بعد از retry خطا داد، دسته به دو نیم تقسیم می‌شود تا متن مشکل‌دار جدا شود. این کار تا رسیدن به یک تک‌متن ادامه می‌یابد و حتی در صورت وجود یک متن نامعتبر، کل فرآیند متوقف نمی‌شود.

۷.۷ کش دیسکی برای Resume

برای پشتیبانی از resume در صورت قطع (مثلاً قطع برق، Ctrl+C، یا خطای شبکه)، یک کش دیسکی پیاده‌سازی شده است. هر دسته امبدینگ، با هش SHA-1 از نام مدل و متن‌های آن دسته، در یک فایل `.npy` ذخیره می‌شود. هنگام اجرای مجدد، اگر کش وجود داشته باشد، بردارها از کش خوانده می‌شوند و هیچ درخواستی به سرویس ارسال نمی‌شود:

```
def batch_cache_path(cache_dir, model, texts):
    h = hashlib.sha1(model.encode("utf-8"))
    for t in texts:
        h.update(hashlib.sha1(t.encode("utf-8")).digest())
    return cache_dir / f"{h.hexdigest()}.npy"


if cpath is not None and cpath.exists():
    try:
        arr = np.load(cpath)
        check_vectors(arr, len(batch), dim)
        cached_batches += 1
    except Exception:
        arr = None
```

```
if arr is None:
 arr = embed_robust(service, batch, dim)
 if cpath is not None:
 tmp = cpath.with_suffix(".tmp.npy")
 np.save(tmp, arr)
 tmp.replace(cpath)
```

این کار باعث می‌شود در صورت قطع اجرا در وسط کار، اجرای بعدی از همان‌جا ادامه یابد و هیچ هزینه امبدینگ مجدد پرداخت نشود. پس از ساخت موفق ایندکس، کش به‌صورت خودکار حذف می‌شود (مگر اینکه با --keep- cache نگه‌داشته شود).

۸.۷ حذف متن‌های تکراری

برای صرفه‌جویی در هزینه امبدینگ، متن‌های تکراری شناسایی و فقط یک‌بار امبد می‌شوند. این کار با ساخت یک دیکشنری seen انجام می‌شود که کلید آن متن و مقدار آن اندیس بردار یکتاست. پس از امبدینگ متن‌های یکتا، بردارها بر اساس اندیس معکوس به ترتیب چانک‌های اصلی بازآرایی می‌شوند:

```
unique_texts: list[str] = []
seen: dict[str, int] = {}
inverse: list[int] = []
for c in chunks:
 idx = seen.get(c.text)
 if idx is None:
 idx = len(unique_texts)
 seen[c.text] = idx
 unique_texts.append(c.text)
 inverse.append(idx)

n_requests = -(-len(unique_texts) // batch_size)
logger.info(" : %d | : %d (%d and ) | and : ~%d",
 len(chunks), len(unique_texts), len(chunks) - len(unique_texts),
 n_requests)


embeddings = unique_emb[np.asarray(inverse)]
```

در یک دیتاست واقعی با ۴۸۰ فایل PDF، معمولاً ۱۰ تا ۲۰ درصد چانک‌ها تکراری هستند (مثلاً صفحات عنوان، تکرار فهرست مطالب در ابتدای هر فایل) که با این روش از امبدینگ مجدد صرف‌نظر می‌شوند.

۹.۷ آمار عملکرد

با تنظیمات پیش‌فرض (دسته ۱۶تایی) و کارت گرافیک NVIDIA RTX 3060، سرعت امبدینگ به‌صورت زیر است:

- سرعت امبدینگ: حدود ۸۰ تا ۱۰۰ چانک در ثانیه
- زمان کل برای ۵۰ هزار چانک: حدود ۸ تا ۱۰ دقیقه
- حجم بردارهای نهایی: حدود ۲۰۰ MB (با ابعاد ۱۰۲۴ و float32)
- حجم ایندکس FAISS: حدود ۲۰۰ MB

این عملکرد برای استقرار محلی کاملاً قابل قبول است و کل فرآیند از جمع‌آوری داده تا ساخت ایندکس آماده، در کمتر از یک ساعت قابل انجام است.

فصل ۸

ذخیره‌گاه برداری FAISS

۱.۸ مقدمه

پس از امبدینگ چانک‌ها، نیاز به یک ذخیره‌گاه هست که بردارها را به‌گونه‌ای ذخیره کند که جست‌وجوی شباهت در آن سریع و دقیق باشد. در این پروژه، از کتابخانه FAISS [۱۸] استفاده شده است که توسط Facebook AI Research توسعه یافته و به‌عنوان استاندارد صنعتی برای جست‌وجوی برداری شناخته می‌شود. این فصل، انتخاب نوع ایندکس، نحوه ساخت، ذخیره و جست‌وجو را شرح می‌دهد.

۲.۸ انتخاب نوع ایندکس: IndexFlatIP

FAISS انواع مختلفی از ایندکس‌ها را ارائه می‌دهد که هر یک تعیینی بین سرعت، دقت و حافظه دارند:

- IndexFlatIP / IndexFlatL2: جست‌وجوی دقیق (brute-force) با محاسبه شباهت با تمام بردارها. دقت ۱۰۰٪ اما سرعت برای حجم‌های بزرگ کند است.
- IVF (Inverted File Index): تقسیم فضای برداری به clusters و جست‌وجو فقط در clusters نزدیک. سرعت بالاتر اما دقت تقریبی.
- HNSW (Hierarchical Navigable Small World): ساخت گراف برای جست‌وجوی تقریبی با سرعت بسیار بالا. مناسب برای حجم‌های بسیار بزرگ.
- IVF-PQ: ترکیب IVF با Product Quantization برای فشرده‌سازی بردارها و کاهش حافظه.

در این پروژه از IndexFlatIP استفاده شده است. دلایل این انتخاب:

۱. **دقت کامل:** برای حجم اسناد IoT (چند صد هزار چانک)، دقت کامل مهم‌تر از سرعت است و جست‌وجوی brute-force با GPU به اندازه کافی سریع است.

۲. **سادگی:** نیازی به آموزش ایندکس (training) ندارد؛ تمام بردارها مستقیماً اضافه می‌شوند.

۳. **امتیاز معادل شباهت کسینوسی:** با نرمال‌سازی L2 در مرحله امبدینگ، امتیاز خروجی IndexFlatIP معادل شباهت کسینوسی است، که معیار استاندارد سنجش شباهت معنایی است.

۴. **بدون نیاز به تنظیم:** پارامترهایی مانند `nlist` یا `nprobe` برای تنظیم نداریم.

۳.۸ ساخت و ذخیره ایندکس

کلاس VectorStore در ماژول `vector_store.py`، ساخت، ذخیره و بارگذاری ایندکس را مدیریت می‌کند. متد build() ایندکس را از چانک‌ها و بردارهای آن‌ها می‌سازد:

```
def build(self, chunks: list[Chunk], embeddings: np.ndarray) -> None:
 """ """
 if not chunks:
 raise IndexNotFoundError("Index not found.")
 embeddings = np.asarray(embeddings, dtype="float32")
 if embeddings.ndim != 2 or embeddings.shape[0] != len(chunks):
 raise ValueError("Invalid input.")
 vectors = np.ascontiguousarray(embeddings)
 dimension = vectors.shape[1]
 index = faiss.IndexFlatIP(dimension)
 index.add(vectors)
 self.index = index
 self.metadatas = [
 {
 "source": chunk.source,
 "chunk_index": chunk.chunk_index,
 "text": chunk.text,
 "char_count": chunk.char_count,
 }
 for chunk in chunks
 ]
 self.info = {
 "dimension": dimension,
 "count": len(chunks),
 "sources": sorted({chunk.source for chunk in chunks}),
 "created_at": time.strftime("%Y-%m-%d %H:%M:%S"),
 }
```

نکته مهم در این کد، استفاده از ascontiguousarray است. FAISS نیاز به آرایه‌های C-contiguous دارد (یعنی در حافظه به‌صورت پیوسته ذخیره شوند). اگر بردارها از عملیاتی مانند transpose یا slicing به‌دست آمده باشند، ممکن است non-contiguous باشند که باعث خطای عجیب در FAISS می‌شود. استفاده از ascontiguousarray این مشکل را حل می‌کند.

۴.۸ ذخیره روی دیسک

برای ماندگاری ایندکس بین اجراها، آن را به دو فایل روی دیسک ذخیره می‌کنیم:

- index.faiss: فایل باینری ایندکس FAISS (شامل تمام بردارها).
- metadata.json: فایل JSON شامل متادیتای هر چانک (نام منبع، شماره چانک، متن کامل، تعداد کاراکتر) و اطلاعات کلی (ابعاد، تعداد، تاریخ ساخت).

```
def save(self) -> Path:
 """ """
 if self.index is None:
 raise IndexNotFoundError("No index has been built to save.")
 self.index_dir.mkdir(parents=True, exist_ok=True)
 faiss.write_index(self.index, str(self.index_dir / self.INDEX_FILE))
 payload = {"info": self.info, "metadatas": self.metadatas}
 (self.index_dir / self.META_FILE).write_text(
 json.dumps(payload, ensure_ascii=False, indent=1), encoding="utf-8"
 )
 return self.index_dir
```

۵.۸ بارگذاری از دیسک

هنگام شروع برنامه، متد `try_load()` بررسی می‌کند که آیا ایندکس از قبل روی دیسک وجود دارد یا خیر. اگر وجود داشت، آن را بارگذاری می‌کند و یک راستی‌آزمایی مهم انجام می‌دهد:

```
def try_load(self) -> bool:
 """"""
 index_path = self.index_dir / self.INDEX_FILE
```

```
meta_path = self.index_dir / self.META_FILE
if not index_path.is_file() or not meta_path.is_file():
return False
try:
self.index = faiss.read_index(str(index_path))
payload = json.loads(meta_path.read_text(encoding="utf-8"))
self.metadatas = payload.get("metadatas", [])
self.info = payload.get("info", {})
if len(self.metadatas) != self.index.ntotal:
raise IndexNotFoundError(
""
""
)
return True
except (OSError, RuntimeError, json.JSONDecodeError, ValueError,
IndexNotFoundError) as exc:
self.index = None
self.metadatas = []
self.info = {}
logger.error("Error: %s", exc)
return False
```

راستی‌آزمایی کلیدی در اینجا، بررسی `self.index.ntotal != len(self.metadatas)` است. اگر تعداد متادیتاها با تعداد بردارهای ایندکس هم‌خوانی نداشته باشد، یعنی فایل ایندکس و فایل متادیتا از دو اجرای متفاوت هستند و باید ایندکس از نو ساخته شود.

۶.۸ جست‌وجوی معنایی

متد search() جست‌وجوی بردار پرس‌وجو را در ایندکس انجام می‌دهد:

```
def search(self, query_vector: np.ndarray, top_k: int) ->
list[RetrievalResult]:
""" """
if not self.is_ready():
return []
vector = np.ascontiguousarray(
query_vector.reshape(1, -1), dtype="float32"
```

```
)
if vector.shape[1] != self.index.d:
raise IndexNotFoundError(
f"Loaded vectors ({vector.shape[1]}) "
f"({self.index.d}) dimensions."
)
k = min(max(1, top_k), self.index.ntotal)
scores, ids = self.index.search(vector, k)
results: list[RetrievalResult] = []
for rank, (score, idx) in enumerate(zip(scores[0].tolist(),
  ids[0].tolist())):
if idx < 0:
continue
meta = self.metadatas[idx]
results.append(RetrievalResult(
text=meta["text"],
score=float(score),
source=meta["source"],
chunk_index=meta["chunk_index"],
rank=rank + 1,
))
return results
```

نکات مهم در این کد:

- بردار پرس‌وجو به `reshape 1- 1,` می‌شود تا ابعاد آن با ورودی مورد انتظار FAISS (دومدوله) مطابقت کند.
- `ntotal) min(top_k, = k` برای جلوگیری از خطای درخواست k بیش از تعداد بردارها.
- بررسی `0 < idx` برای مدیریت موقعی که FAISS برای k بیش از ntotal، مقدار `1-` برمی‌گرداند.

۷.۸ آمار ایندکس

متد stats() اطلاعات کلی ایندکس را برای نمایش در رابط کاربری و ابزار `get_knowledge_base_stats` برمی‌گرداند:

- `ready`: آیا ایندکس بارگذاری شده و حداقل یک بردار دارد؟
- `count`: تعداد چانک‌ها در ایندکس.

- `dimension`: ابعاد بردارها (۱۰۲۴ برای bge-m3).
- `sources`: فهرست نام فایل‌های موجود در دانش‌نامه.
- `created_at`: تاریخ و زمان ساخت ایندکس.

این اطلاعات در سایدبار رابط کاربری نمایش داده می‌شوند تا کاربر وضعیت دانش‌نامه را ببیند.

۸.۸ مقایسه با سایر پایگاه‌های داده برداری

جدول ۱.۸ مقایسه‌ای بین FAISS و سایر گزینه‌های رایج ارائه می‌دهد:

جدول ۱.۸: مقایسه پایگاه‌های داده برداری.

ویژگی	FAISS	Milvus	Weaviate	Qdrant
نوع	کتابخانه	سرور توزیع‌شده	سرور مستقل	سرور مستقل
استقرار	داخل کد	جداگانه	جداگانه	جداگانه
مقیاس	میلیون‌ها	میلیاردها	میلیون‌ها	میلیون‌ها
پشتیبانی GPU	بله	بله	خیر	خیر
فیلتر متادیتا	دستی	بله	بله	بله
سادگی	بسیار ساده	متوسط	متوسط	ساده

همان‌طور که مشاهده می‌شود، FAISS برای حجم‌های متوسط و استقرار ساده، بهترین انتخاب است. اگر پروژه به مقیاس میلیاردی برسد یا نیاز به فیلتر پیشرفته متادیتا باشد، می‌توان به Milvus یا Qdrant مهاجرت کرد.

فصل ۹

استقرار با Docker و llama.cpp

۱.۹ مقدمه

برای اجرای مدل‌های زبانی و امبدینگ به‌صورت سرویس‌دهی واقعی روی سخت‌افزار کاربر، از دو فناوری کلیدی استفاده شده است: Docker [۳۸] به‌عنوان پلتفرم کانتینری‌سازی، و llama.cpp [۳۹] به‌عنوان موتور استنتاج بهینه‌شده برای اجرای مدل‌های GGUF روی CPU و GPU. این ترکیب، چندین مزیت کلیدی فراهم می‌کند: ایزوله‌سازی محیط اجرا (که از تداخل با سایر نرم‌افزارهای سیستم جلوگیری می‌کند)، قابلیت تکرارپذیری (که همان نتیجه را روی هر سیستمی تضمین می‌کند)، و پشتیبانی از GPU با NVIDIA Container Toolkit [۴۰]. این فصل، پیکربندی کامل دو کانتینر مجزا برای مدل Qwen3-4B و امبدینگ bge-m3 را به‌تفصیل شرح می‌دهد.

۲.۹ معماری دو کانتینری

سیستم از دو کانتینر مجزا تشکیل شده است که هر کدام یک مدل را سرو می‌کنند. کانتینر اول با نام `qwen3` مدل زبانی Qwen3-4B با کوانتایز `Q4_K_M` را روی پورت ۸۰۸۰ سرو می‌کند. کانتینر دوم با نام `bge-m3` مدل امبدینگ bge-m3 با وزن F16 را روی پورت ۸۰۸۱ سرو می‌کند. هر دو کانتینر از یک تصویر پایه به نام `llama-cuda13:local` استفاده می‌کنند که llama.cpp را با پشتیبانی از CUDA 13 و کارت گرافیک NVIDIA شامل می‌شود.

این جداسازی دو دلیل عمده دارد. نخست، دو مدل با هدف کاملاً متفاوت (تولید متن در مقابل امبدینگ) باید با پارامترهای متفاوت سرو شوند؛ مثلاً مدل زبانی به context window بزرگ‌تر نیاز دارد در حالی که مدل امبدینگ از batch processing فعال استفاده می‌کند. دوم، با جداسازی، می‌توان هر کانتینر را مستقلاً به‌روزرسانی، راه‌اندازی مجدد یا متوقف کرد بدون آنکه دیگری تحت تأثیر قرار گیرد. این ویژگی در محیط توسعه بسیار ارزشمند است.

۳.۹ کانتینر مدل زبانی Qwen3-4B

پیکربندی کامل کانتینر مدل زبانی در فایل docker-compose.yml به شرح زیر است:

```yaml
services:

 llm:
 image: llama-cuda13:local
 container_name: qwen3
 restart: unless-stopped

 ports:
 - "8080:8080"

 volumes:
 - /mnt/d/AI/models:/models:ro

 command:
 - -m
 - /models/Qwen_Qwen3-4B-Q4_K_M.gguf
 - --host
 - 0.0.0.0
 - --port
 - "8080"
 - -ngl
 - "99"
 - -c
 - "8192"
 - -np
 - "1"
 - --reasoning
 - "off"
 - -lv
 - "3"

 deploy:
 resources:
 reservations:
 devices:
 - driver: nvidia
 count: all
 capabilities: [gpu]
```

پارامترهای کلیدی در این پیکربندی عبارتند از:

- `-m /models/Qwen_Qwen3-4B-Q4_K_M.gguf`: مسیر فایل مدل با فرمت GGUF و کوانتایز Q4_K_M. این فایل حدود ۲٫۵ گیگابایت حجم دارد.
- `--host 0.0.0.0`: گوش دادن روی تمام رابط‌های شبکه برای دسترسی از بیرون کانتینر. این برای ارتباط پروژه پایتون با کانتینر حیاتی است.
- `--port 8080`: پورت سرویس مدل زبانی.
- `-ngl 99`: قرار دادن تمام لایه‌های مدل روی GPU برای استنتاج سریع. عدد ۹۹ به‌معنای «همه لایه‌ها» است.
- `-c 8192`: اندازه پنجره زمینه ۸۱۹۲ توکن. این مقدار برای پاسخ‌دهی به سوالات تخصصی با بافت طولانی مناسب است و در عین حال از مصرف بیش از حد حافظه جلوگیری می‌کند.
- `-np 1`: تعداد اسلات‌های موازی برای درخواست‌ها. برای کاربرد تک‌کاربره، ۱ کافی است.
- `--reasoning off`: غیرفعال‌سازی حالت استدلال داخلی مدل Qwen3. در این پروژه، استدلال از طریق Tool Calling انجام می‌شود و حالت داخلی غیرفعال می‌باشد.
- `-lv 3`: سطح لاگ verbose برای دیباگ.

۱.۳.۹ چرا کوانتایز Q4_K_M؟

مدل Qwen3-4B در نسخه‌های مختلف کوانتایز در دسترس است: F16 (دقیق‌ترین، ۸ گیگابایت)، Q8_0 (۴ گیگابایت)، Q4_K_M (۲٫۵ گیگابایت) و Q4_0 (۲ گیگابایت). انتخاب Q4_K_M بر اساس سه معیار بود:

۱. **کیفیت:** Q4_K_M (نسخه K-quant متوسط) کیفیت نزدیک به Q8_0 دارد اما حجم آن کمتر است. در آزمایش‌های عملی، تفاوت کیفیت پاسخ بین Q4_K_M و F16 ناچیز است و در اکثر سناریوهای کاربردی قابل صرف‌نظر است.

۲. **حافظه:** با ۲٫۵ گیگابایت حجم، این مدل روی کارت گرافیک با ۸ گیگابایت حافظه به‌راحتی اجرا می‌شود و فضای کافی برای پنجره زمینه، بافت گفتگو و KV cache باقی می‌ماند.

۳. **سرعت:** کوانتایز پایین‌تر، استنتاج سریع‌تر دارد. Q4_K_M حدود ۳۰ تا ۴۰ توکن در ثانیه روی RTX 3060 تولید می‌کند که برای کاربرد چت کاملاً کافی است و تجربه روان کاربری فراهم می‌کند.

۴.۹ کانتینر امبدینگ bge-m3

پیکربندی کانتینر امبدینگ در فصل ۷ به‌تفصیل شرح داده شد. در اینجا، تفاوت‌های کلیدی آن با کانتینر مدل زبانی را مرور می‌کنیم:

- **وزن F16:** برای امبدینگ، دقت اهمیت بیشتری از سرعت دارد. F16 بهترین تعادل را فراهم می‌کند و افت کیفیت ناشی از کوانتایز پایین‌تر در امبدینگ‌ها می‌تواند به‌طور قابل توجهی دقت بازیابی را کاهش دهد.
- **حالت embedding:** با پرچم `--embedding` فعال می‌شود. این پرچم به llama.cpp می‌گوید که به‌جای تولید متن، بردار امبدینگ تولید کند.
- **Pooling cls:** با `--pooling cls` تنظیم می‌شود. این روش، خروجی توکن [CLS] را به‌عنوان بردار نهایی استفاده می‌کند که بهترین روش برای bge-m3 است.
- **۴ اسلات موازی:** با `-np 4`، چهار درخواست امبدینگ به‌صورت هم‌زمان قابل پردازش است. این برای پردازش دسته‌ای حیاتی است و سرعت امبدینگ مجموعه‌های بزرگ را به‌طور قابل توجهی افزایش می‌دهد.
- **Flash Attention:** با `-fa on` فعال می‌شود که مصرف حافظه را کاهش و سرعت را افزایش می‌دهد. این بهینه‌سازی برای امبدینگ متن‌های بلند بسیار مهم است.

۵.۹ تخصیص GPU با NVIDIA Container Toolkit

برای دسترسی کانتینرها به کارت گرافیک، از NVIDIA Container Toolkit [۴۰] استفاده می‌شود. این ابزار، درایور NVIDIA را به کانتینرها تخصیص می‌دهد و امکان اجرای مستقیم کد CUDA داخل کانتینر را فراهم می‌کند. بخش `deploy` در docker-compose.yml این تخصیص را انجام می‌دهد:

```
deploy:
 resources:
 reservations:
 devices:
 - driver: nvidia
 count: all
 capabilities: [gpu]
```

Listing 9.1: GPU allocation to containers.

این پیکربندی به کانتینر اجازه می‌دهد از تمام GPUهای موجود استفاده کند. اگر چند کارت گرافیک داشته باشیم، می‌توان با `device_ids` به‌جای `count: all`، کارت خاصی را اختصاص داد. همچنین، می‌توان با تنظیم `count: 1`، تعداد GPUهای قابل استفاده را محدود کرد.

۶.۹ بررسی سلامت سرویس‌ها

اسکریپت `check_services.py` در پوشه `scripts/`، سلامت هر دو سرویس را بررسی می‌کند و در صورت بروز مشکل، راه‌حل پیشنهاد می‌دهد. این اسکریپت با ارسال درخواست GET /health به هر سرویس، کد وضعیت ۲۰۰ را بررسی می‌کند و در صورت عدم دسترسی، پیام راهنمایی برای `docker compose up -d` نمایش می‌دهد:

```
python scripts/check_services.py
```

۷.۹ رابط API سازگار با OpenAI

یکی از مزایای کلیدی llama.cpp، ارائه API سازگار با OpenAI [۴۱] است. این یعنی ما می‌توانیم از همان ساختار درخواست/پاسخ OpenAI استفاده کنیم:

- `POST /v1/chat/completions`: ارسال پیام و دریافت پاسخ (با یا بدون stream).
- `POST /v1/embeddings`: ارسال متن و دریافت بردار امبدینگ.
- `GET /health`: بررسی سلامت سرویس.

این سازگاری باعث می‌شود کد پایتون پروژه بتواند با همان کتابخانه‌هایی که برای OpenAI API استفاده می‌شوند، با سرویس محلی نیز کار کند. در این پروژه، از کتابخانه `requests` به‌صورت مستقیم استفاده شده است تا وابستگی به کتابخانه `openai` حذف شود و کنترل کامل روی جریان SSE (Server-Sent Events) برای استریم داشته باشیم.

۸.۹ سرور آزمایشی برای توسعه آفلاین

برای توسعه و تست بدون نیاز به GPU یا سرویس‌های واقعی، اسکریپت `mock_server.py` یک سرور آزمایشی را شبیه‌سازی می‌کند که تمام endpointها (شامل رفتار فراخوانی ابزار) را تقلید می‌کند. این سرور برای توسعه رابط کاربری، تست پارسر Tool Calling و آزمایش جریان استریم بسیار مفید است؛ زیرا نیازی به اجرای مدل واقعی ندارد و سرعت توسعه را به‌طور قابل توجهی افزایش می‌دهد:

```
python scripts/mock_server.py --port 8090
```

```
python scripts/embed_dataset.py --data data/data.csv
python run_app.py
```

۹.۹ جمع‌بندی

استقرار با Docker و llama.cpp یک راه‌حل پایدار، قابل تکرار و کارآمد برای سرویس‌دهی مدل‌های زبانی و امبدینگ فراهم می‌کند. دو کانتینر مجزا با پیکربندی بهینه، روی کارت گرافیک مصرفی اجرا می‌شوند و از طریق API سازگار با OpenAI با پروژه پایتون ارتباط برقرار می‌کنند. این معماری اجازه می‌دهد کل سیستم روی یک کامپیوتر شخصی با کارت گرافیک متوسط اجرا شود، بدون نیاز به سرویس‌های ابری خارجی و بدون ارسال هیچ داده‌ای به خارج از محیط کاربر.

فصل ۱۰

پایپلاین RAG نسخه ابزار-محور

۱.۱۰ مقدمه

پایپلاین RAG [۷، ۱۱]، قلب سیستم است که اجزای مختلف (مدل زبانی، ابزارها، تاریخچه، تنظیمات) را هماهنگ می‌کند و برای هر پرسش کاربر، یک پاسخ نهایی تولید می‌کند. در این پروژه، یک نسخه خاص از RAG پیاده‌سازی شده است که آن را «ابزار-محور» [1] می‌نامیم. تفاوت کلیدی این نسخه با RAG کلاسیک این است که هیچ امبدینگ یا جست‌وجوی خودکاری در ابتدای هر پرسش انجام نمی‌شود؛ بلکه خود مدل زبانی تصمیم می‌گیرد که چه زمانی به دانش‌نامه نیاز دارد و از طریق Tool Calling آن را فراخوانی می‌کند. این فصل، معماری و پیاده‌سازی این پایپلاین را به‌تفصیل شرح می‌دهد.

۲.۱۰ تفاوت با RAG کلاسیک

در RAG کلاسیک، فرآیند پاسخ‌دهی به این صورت است: ابتدا پرسش کاربر با مدل امبدینگ به بردار تبدیل می‌شود؛ سپس این بردار در FAISS جست‌وجو می‌شود و top-k چانک‌های مرتبط بازیابی می‌شوند؛ این چانک‌ها در پرامپت مدل قرار می‌گیرند؛ و در نهایت مدل بر اساس بافت و چانک‌ها، پاسخ تولید می‌کند. این رویکرد برای سوالات تخصصی کاربرد دارد، اما برای سوالات عمومی مانند «سلام» یا «ممنون» نیز انجام می‌شود که بی‌فایده است و منابع محاسباتی را هدر می‌دهد. علاوه بر این، در این رویکرد، عبارت جست‌وجو همان پرسش کاربر است، در حالی که مدل زبانی معمولاً می‌تواند عبارت جست‌جوی بهتری بسازد (مثلاً استخراج کلمات کلیدی یا تبدیل به انگلیسی).

در نسخه ابزار-محور، فرآیند به این صورت تغییر می‌کند: پیام کاربر مستقیماً به مدل ارسال می‌شود (بدون هیچ امبدینگ یا جست‌جویی). مدل خود تصمیم می‌گیرد که آیا به دانش‌نامه نیاز دارد یا خیر. اگر سوال عمومی است (مثل سلام‌واحوال‌پرسی)، مستقیماً پاسخ می‌دهد. اگر سوال به اسناد مربوط است، یک بلوک `<tool_call>` در خروجی خود تولید می‌کند. اگر مدل ابزار را فراخوانی کرد، پارسر بلوک را تشخیص می‌دهد، ابزار اجرا می‌شود (شامل امبدینگ پرس‌وجو و جست‌وجوی FAISS)، نتایج به مدل برمی‌گردند، و مدل با توجه به نتایج، پاسخ نهایی را تولید می‌کند.

[1] Tool-Call-First

این رویکرد دو مزیت کلیدی دارد. نخست، برای سوالات عمومی هیچ منابعی هدر نمی‌رود؛ مثلاً اگر کاربر بگوید «سلام»، هیچ درخواستی به سرویس امبدینگ ارسال نمی‌شود. دوم، مدل می‌تواند عبارت جست‌جوی بهتری بسازد؛ مثلاً به‌جای جست‌جوی کل سوال «حداکثر طول بسته در CoAP چقدر است؟»، فقط عبارت «CoAP MTU payload size» را جست‌جو کند که نتایج مرتبط‌تری برمی‌گرداند.

۳.۱۰ کلاس RAGPipeline

کلاس RAGPipeline در ماژول `rag_pipeline.py`، هماهنگ‌کننده اجزای سیستم است. این کلاس بسیار ساده طراحی شده و فقط وابستگی‌ها را نگه می‌دارد:

```
class RAGPipeline:
 """"""

 def __init__(self, llm: LLMClient, tools: ToolRegistry, settings:
   Settings):
 self.llm = llm
 self.tools = tools
 self.settings = settings

 def create_run(self, question: str, history: list[dict] | None =
   None,
 temperature: float | None = None) -> PipelineRun:
 """Create a pipeline run for the given question."""
 return PipelineRun(self, question, history or [], temperature)
```

همان‌طور که مشاهده می‌شود، این کلاس فقط سه وابستگی دارد: LLMClient برای ارتباط با مدل، ToolRegistry برای اجرای ابزارها، و Settings برای پارامترها. نکته مهم این است که Retriever به‌صورت مستقیم در این کلاس استفاده نمی‌شود؛ جست‌وجوی معنایی فقط از طریق ToolRegistry و با فراخوانی خود مدل اتفاق می‌افتد. این جداسازی، اصل Single Responsibility را رعایت می‌کند و وابستگی‌های کد را کاهش می‌دهد.

۴.۱۰ کلاس PipelineRun

کلاس PipelineRun یک اجرای خاص پایپلاین را برای یک سوال مشخص نگه می‌دارد. این کلاس قابلیت استریم (نمایش تدریجی پاسخ) را فراهم می‌کند و نتیجه کامل را در `self.result` نگه می‌دارد. متد اصلی این کلاس، stream() است که یک مولد (generator) است و توکن‌های پاسخ را به‌تدریج برمی‌گرداند:

```
def stream(self) -> Iterator[str]:
 """""""
 settings = self.pipeline.settings

 messages: list[dict] = [{"role": "system", "content":
   build_system_prompt()}]
 messages.extend(self.history)
 messages.append({"role": "user", "content": self.question})


 tool_names = tuple(self.pipeline.tools.tool_names())
 display_parts: list[str] = []
 total_rounds = settings.max_tool_calls + 1
 for round_number in range(1, total_rounds + 1):
 forced_final = round_number == total_rounds
 self.emit(EVENT_LLM_ROUND, round=round_number,
   forced_final=forced_final)
 if forced_final:
 messages.append({
 "role": "system",
 "content": " and "
 "",
 })


 mask = StreamMaskFilter(tool_names=tool_names)
 collected: list[str] = []
 visible_text = ""
 was_inside_block = False
 for token in self.pipeline.llm.chat_stream(
 messages, temperature=self.temperature,
 max_tokens=settings.max_tokens,
 ):
 collected.append(token)
 piece = mask.feed(token)
 inside_block = mask.inside_block
 if inside_block != was_inside_block:
 self.emit(EVENT_THINKING, active=inside_block)
 was_inside_block = inside_block
```

```
if piece:
visible_text += piece
yield piece
if was_inside_block:
self.emit(EVENT_THINKING, active=False)
tail = mask.flush()
if tail:
visible_text += tail
yield tail
raw_text = "".join(collected)


calls = ([] if forced_final
else parse_tool_calls(raw_text, tool_names=tool_names))
if not calls:
self.result.answer = self._compose_answer(
display_parts, visible_text, raw_text, tool_names
)
self.result.finished = True
return


messages.append({"role": "assistant", "content": raw_text})
for call in calls:
self.result.tool_calls.append(call)
self.emit(EVENT_TOOL_CALL, name=call.name, arguments=call.arguments)
execution = self.pipeline.tools.execute(call)
self.result.sources.extend(execution.sources)
summary = execution.text[:160].replace("\n", " ")
self.emit(EVENT_TOOL_RESULT, name=call.name, summary=summary)
messages.append({
"role": "user",
"content": build_tool_response_message(call.name, execution.text),
})
if visible_text.strip():
display_parts.append(visible_text.strip())

self.result.answer = self._compose_answer(display_parts, "", "",
  tool_names)
self.result.finished = True
```

۵.۱۰ حلقه مدل–ابزار

حلقه اصلی پایپلاین، یک چرخه مدل–ابزار است که حداکثر `MAX_TOOL_CALLS = 3` بار تکرار می‌شود. هر دور شامل مراحل زیر است:

۱. **دور تولید مدل:** متن پاسخ مدل به‌صورت استریم دریافت می‌شود. در حین دریافت، StreamMaskFilter بلوک‌های <tool_call> و <think> را فیلتر می‌کند و فقط متن قابل‌نمایش را به کاربر نشان می‌دهد.

۲. **تشخیص فراخوانی ابزار:** پس از اتمام تولید، متن خام کامل با `parse_tool_calls` بررسی می‌شود. اگر بلوک <tool_call> وجود داشت، آن استخراج می‌شود.

۳. **اجرای ابزار:** ابزار با آرگومان‌های مشخص‌شده اجرا می‌شود. مثلاً `search_knowledge_base` با Retriever.search اجرا می‌شود که شامل امبدینگ پرس‌وجو و جست‌وجوی FAISS است.

۴. **بازگشت نتیجه به مدل:** نتیجه ابزار در قالب بلوک <tool_response> به مدل برمی‌گردد و دور جدید تولید شروع می‌شود.

۵. **پایان:** اگر مدل ابزار را فراخوانی نکرد، یا به حداکثر تعداد فراخوانی رسید، حلقه پایان می‌یابد و پاسخ نهایی برگردانده می‌شود.

۶.۱۰ رویدادهای پایپلاین

پایپلاین پنج نوع رویداد را از طریق callback به رابط کاربری ارسال می‌کند تا وضعیت زنده نمایش داده شود:

جدول ۱.۱۰: رویدادهای پایپلاین برای رابط کاربری.

رویداد	توضیح
`EVENT_TOOL_CALL`	مدل ابزاری را فراخوانی کرد (نام، آرگومان‌ها)
`EVENT_TOOL_RESULT`	نتیجه ابزار آماده شد (نام، خلاصه)
`EVENT_LLM_ROUND`	دور جدید تولید (شماره دور، آیا اجباری است)
`EVENT_THINKING`	مدل وارد/خارج بلوک تفکر شد (فعال/غیرفعال)
`EVENT_WARNING`	هشدار غیرمهلک (پیام)

این رویدادها در رابط کاربری به‌عنوان نوار وضعیت نمایش داده می‌شوند: «در حال تفکر...»، «در حال جست‌وجو در دانش‌نامه...»، «در حال نوشتن پاسخ...». این کار تجربه کاربری روان و شفافی فراهم می‌کند و کاربر می‌بیند که سیستم در چه مرحله‌ای است.

۷.۱۰ مدیریت خطا

پایپلاین با بلاک try/except خطاهای احتمالی را مدیریت می‌کند. اگر خطایی رخ دهد (مثلاً اتصال به مدل قطع شود)، یک پیام خطای کاربرپسند به کاربر نمایش داده می‌شود:

```
except Exception as exc:
 logger.exception("Invalid input.")
 self.result.error = f" and : {exc}"
 self.result.finished = False
 yield ("\n\n(Connection to model failed or error in response
   generation; "
 "check service health with scripts/check_services.py.)")
```

این مدیریت خطا تضمین می‌کند که در صورت بروز مشکل، رابط کاربری قفل نمی‌شود و کاربر یک راهنمایی واضح برای رفع مشکل دریافت می‌کند. همچنین، خطا در logger.exception ثبت می‌شود تا دیباگ آسان‌تر باشد.

۸.۱۰ حداکثر فراخوانی ابزار

پارامتر MAX_TOOL_CALLS = 3 حداکثر تعداد فراخوانی ابزار در یک پاسخ را تعیین می‌کند. این محدودیت دو دلیل دارد: نخست، جلوگیری از حلقه بی‌نهایت (که در آن مدل مدام ابزار را فراخوانی کند بدون رسیدن به پاسخ)؛ و دوم، کنترل هزینه محاسباتی و زمان پاسخ. اگر مدل سه بار ابزار را فراخوانی کند و هنوز به پاسخ نرسد، در دور چهارم یک پیام سیستمی به مدل ارسال می‌شود که «دیگر فرصتی برای فراخوانی ابزار نیست» و مدل باید بر اساس اطلاعات موجود، بهترین پاسخ ممکن را تولید کند.

فصل ۱۱

سیستم Tool Calling و Function Calling

۱.۱۱ مقدمه

Tool Calling یا Function Calling [۲۴، ۲۲، ۲۵]، قابلیتی است که به مدل زبانی اجازه می‌دهد در طول تولید پاسخ، توابع خارجی را فراخوانی کند و از خروجی آن‌ها برای ادامه تولید استفاده کند. این قابلیت در پروژه حاضر برای پیاده‌سازی معماری ابزار–محور حیاتی است: مدل خود تصمیم می‌گیرد چه زمانی به دانش‌نامه نیاز دارد و ابزار `search_knowledge_base` را فراخوانی می‌کند. این فصل، تعریف ابزارها، آموزش قالب فراخوانی به مدل، پارسر مقاوم برای تشخیص فراخوانی‌ها و فیلتر استریم را شرح می‌دهد.

۲.۱۱ تعریف ابزارها

دو ابزار در اختیار مدل قرار گرفته شده است:

۱. **`search_knowledge_base(query, top_k)`:** جست‌وجوی معنایی در دانش‌نامه محلی (ایندکس FAISS). هر زمان که پاسخ سوال کاربر در بافت ارائه‌شده نیست یا به اطلاعات بیشتری نیاز است، یک عبارت جست‌جوی مناسب ساخته و این ابزار فراخوانی می‌شود. پارامتر `query` عبارت جست‌جو (ترجیحاً به زبان محتوای اسناد) و `top_k` تعداد نتایج (بین ۱ تا ۱۵، پیش‌فرض ۵) است.

۲. **`get_knowledge_base_stats()`:** آمار دانش‌نامه (تعداد فایل‌ها، تعداد چانک‌ها، ابعاد بردارها). این ابزار بدون پارامتر است و فقط برای اطلاع از وضعیت دانش‌نامه استفاده می‌شود.

تعریف این ابزارها با اسکیمای OpenAI در ماژول `tools.py` پیاده‌سازی شده است:

```
TOOL_DEFINITIONS: list[dict] = [
 {
 "type": "function",
 "function": {
```

```
 "name": "search_knowledge_base",
 "description": (
 "Search the Knowledge Base (FAISS). "
 ""
 ""
 ),
 "parameters": {
 "type": "object",
 "properties": {
 "query": {
 "type": "string",
 "description": "Search for relevant documents.",
 },
 "top_k": {
 "type": "integer",
 "description": " 1 15 ( 5)",
 },
 },
 "required": ["query"],
 },
 },
 },
 {
 "type": "function",
 "function": {
 "name": "get_knowledge_base_stats",
 "description": "Knowledge Base: get statistics.",
 "parameters": {"type": "object", "properties": {}, "required": []},
 },
 },
]
```

۳.۱۱ آموزش قالب Hermes به مدل

مدل Qwen3 با قالب Hermes [۲۳] سازگار است، اما برای اطمینان از تولید صحیح بلوک‌های فراخوانی، آموزش دقیقی در پرامپت سیستمی ارائه شده است. این آموزش در تابع `render_tool_instructions()` از ماژول `prompts.py` ساخته می‌شود:

```python
def render_tool_instructions() -> str:
    """Introduce tools to model in hermes style (Qwen3 compatible)."""
    lines: list[str] = [
    "# Tools",
    "",
    "You can use the following tools to get more information.",
    "",
    "## Call format (very important - exactly this structure)",
    f"Always write tool call inside tags {TOOL_CALL_OPEN} and {TOOL_CALL_CLOSE} like this:",
    "",
    TOOL_CALL_OPEN,
    '{"name": "tool_name", "arguments": {"param": "value"}}',
    TOOL_CALL_CLOSE,
    "",
    "Rules:",
    f"- Only one tool call per turn between {TOOL_CALL_OPEN} and {TOOL_CALL_CLOSE} tags.",
    "- Output valid JSON with name and arguments fields.",
    "- Wait for tool result before continuing.",
    "Result of tool call will be provided in next message.",
    "- If tool result is JSON, parse it and use the content.",
    "",
    "Available tools:",
    "",
    ]
    for tool in TOOL_DEFINITIONS:
    function = tool["function"]
    params = function.get("parameters", {}).get("properties", {})
    param_names = ", ".join(params) if params else "none"
    lines.append(f"- {function['name']}({param_names}): {function['description']}")
    return "\n".join(lines)
```

علاوه بر این آموزش، پرامپت سیستمی شامل قواعد استفاده از ابزار است که به مدل می‌گوید چه زمانی باید ابزار را فراخوانی کند و چه زمانی نباید. این قواعد در چهار مورد خلاصه می‌شوند: اول، فقط وقتی ابزار را فراخوانی کن که واقعاً به اطلاعات دانش‌نامه نیاز داری. دوم، در هر نوبت حداکثر یک فراخوانی؛ سپس منتظر نتیجه بمان. سوم، هرگز

خروجی ساختگی برای ابزار نساز؛ نتیجه در پیام بعدی به تو داده می‌شود. چهارم، اگر نتایج جست‌وجو به سوال مربوط نبود یا ناکافی بود، می‌توانید حداکثر دو بار دیگر با عبارت متفاوت و بهتری جست‌وجو کنید.

۴.۱۱ پارسر مقاوم parse_tool_calls

در عمل، مدل Qwen3 گاهی فراخوانی ابزار را «بدون تگ» و به‌صورت یک JSON خالی در میان متن فارسی می‌نویسد (مثلاً بعد از یک جمله مقدمه‌ای). برای پشتیبانی از هر دو حالت، یک پارسر مقاوم در ماژول tool_parsing.py پیاده‌سازی شده است:

```
def parse_tool_calls(text: str, tool_names=None) -> list[ToolCall]:
 """Extract all tool calls from model raw text (tagged untagged)."""
 calls: list[ToolCall] = []


 for match in TOOL_CALL_BLOCK_RE.finditer(text):
 payload = _safe_json_loads(match.group(1))
 if payload is None or not _is_tool_call_payload(payload, tool_names):
 continue
 calls.append(ToolCall(name=str(payload["name"]).strip(),
 arguments=_extract_arguments(payload),
 raw=match.group(0)))


 remainder = TOOL_CALL_BLOCK_RE.sub("", text)
 remainder = THINK_BLOCK_RE.sub("", remainder)
 remainder = remainder.replace(TOOL_CALL_OPEN,
   "").replace(TOOL_CALL_CLOSE, "")
 for match in _iter_balanced_json_objects(remainder):
 obj_text = remainder[match[0]:match[1]]
 payload = _safe_json_loads(obj_text)
 if payload is None or not _is_tool_call_payload(payload, tool_names):
 continue
 calls.append(ToolCall(name=str(payload["name"]).strip(),
 arguments=_extract_arguments(payload),
 raw=obj_text))
 return calls
```

تابع `_iter_balanced_json_objects` با پیمایش متوازن آکولادها، حتی JSONهای وسط متن فارسی را تشخیص می‌دهد. این کار با در نظر گرفتن حالت string (با کوتیشن) انجام می‌شود تا آکولادهای داخل رشته‌ها نادیده گرفته شوند. این پارسر بسیار مهم است؛ زیرا بدون آن، فراخوانی‌های ابزار در مواردی که مدل قالب را دقیقاً رعایت نکرده، تشخیص داده نمی‌شدند و سیستم از کار می‌افتاد.

۵.۱۱ فیلتر استریم StreamMaskFilter

در حالت استریم (نمایش زنده پاسخ توکن‌به‌توکن)، نیاز است که بلوک‌های `<tool_call>` و <think> از نمایش کاربر پنهان شوند، حتی اگر بین چند توکن شکسته شده باشند. این کار با کلاس StreamMaskFilter انجام می‌شود:

```
class StreamMaskFilter:
 """"""

 OPENINGS = (
 ("think_open", "think_close"),
 (TOOL_CALL_OPEN, TOOL_CALL_CLOSE),
 )

 def __init__(self, tool_names=None):
 self._buffer = ""
 self._closing_tag: str | None = None
 self._done = False
 self._tool_names = {name.strip() for name in (tool_names or ())}

 self._json_depth = 0
 self._json_in_string = False
 self._json_escaped = False
 self._json_active = False

 @property
 def inside_block(self) -> bool:
 """Is stream currently inside a hidden region (thinking/tool
   call)?"""
 return self._closing_tag is not None or self._json_active

 def feed(self, chunk: str) -> str:
 """Receive a chunk from model output; return displayable part."""
```

ویژگی `inside_block` به‌طور زنده به رابط کاربری اطلاع می‌دهد که آیا مدل در حال تفکر است یا در حال نوشتن پاسخ. این کار به کاربر تجربه بصری بهتری می‌دهد: وقتی مدل در حال تفکر است، نوار وضعیت «در حال تفکر...» نمایش داده می‌شود و وقتی خارج از بلوک تفکر است، «در حال نوشتن پاسخ...».

۶.۱۱ کلاس ToolRegistry

کلاس ToolRegistry مسئول ثبت، اعتبارسنجی و اجرای فراخوانی ابزارها توسط مدل است. این کلاس با گرفتن یک ToolCall (شامل نام ابزار و آرگومان‌ها)، ابزار مناسب را پیدا کرده و اجرا می‌کند:

```
class ToolRegistry:
 """"""

 def __init__(self, retriever: Retriever, vector_store: VectorStore,
 tool_result_max_chars: int = 3000, chunk_preview_chars: int = 1200,
 default_top_k: int = 5):
 self.retriever = retriever
 self.vector_store = vector_store
 self.tool_result_max_chars = tool_result_max_chars
 self.chunk_preview_chars = chunk_preview_chars
 self.default_top_k = max(1, int(default_top_k))

 def execute(self, call: ToolCall) -> ToolExecutionResult:
 """" """"
 handler = getattr(self, f"_run_{call.name}", None)
 if handler is None:
 return ToolExecutionResult(text=f"Unknown tool: {call.name}")
 try:
 return handler(**call.arguments) if call.arguments else handler()
 except TypeError as exc:
 return ToolExecutionResult(
 text=f"Error in {call.name}: {exc}"
 )
 except Exception as exc:
 return ToolExecutionResult(text=f" «{call.name}»: {exc}")
```

نکته مهم در این طراحی: خطای هر ابزار به‌صورت متن خطا به مدل برمی‌گردد، تا مدل بتواند آن را به کاربر توضیح

دهد یا مسیر دیگری را امتحان کند. این کار باعث می‌شود خطای یک ابزار، کل پایپلاین را متوقف نکند و سیستم مقاوم‌تر باشد.

۷.۱۱ ابزار search_knowledge_base

ابزار اصلی جست‌وجو، با اجرای Retriever.search و قالب‌بندی نتایج برای مدل:

```
def _run_search_knowledge_base(self, query: str = "", top_k: int |
    None = None
) -> ToolExecutionResult:
query = str(query).strip()
if not query:
return ToolExecutionResult(text="Error: query parameter is empty.")
chosen = self.default_top_k if top_k is None else top_k
top_k = self._clamp_int(chosen, 1, 15, default=self.default_top_k)
if not self.vector_store.is_ready():
return ToolExecutionResult(
text="Knowledge Base loaded (FAISS)."
)
results: list[RetrievalResult] = self.retriever.search(query, top_k)
if not results:
return ToolExecutionResult(text="Index not found.")
blocks: list[str] = []
for item in results:
preview = item.text[: self.chunk_preview_chars]
blocks.append(
f"[{item.rank}] : {item.source} | : {item.score:.2f}\n{preview}"
)
text = "\n\n".join(blocks)
if len(text) > self.tool_result_max_chars:
text = text[: self.tool_result_max_chars] + "\n...( )"
return ToolExecutionResult(text=text, sources=results)
```

خروجی این ابزار، متنی است که به مدل داده می‌شود. هر نتیجه شامل رتبه، نام منبع، امتیاز شباهت و پیش‌نمایش متن (تا ۱۲۰۰ کاراکتر) است. اگر مجموع نتایج از سقف `tool_result_max_chars = 3000` فراتر رود، متن بریده می‌شود تا در پرامپت مدل جا شود.

۸.۱۱ چرا بدون jinja-- کار می‌کند؟

llama.cpp از نسخه‌های اخیر، قابلیت `jinja--` را برای پشتیبانی از قالب‌های گفتگوی پیچیده ارائه می‌دهد. با این حال، در این پروژه از این قابلیت استفاده نشده است و خودمان خروجی مدل را parse می‌کنیم. دلایل این انتخاب:

۱. **سازگاری با نسخه‌های قدیمی‌تر:** پارسر اختصاصی روی هر نسخه llama.cpp کار می‌کند، حتی نسخه‌هایی که `jinja--` را پشتیبانی نمی‌کنند.

۲. **کنترل کامل:** با پارسر اختصاصی، می‌توانیم حالت‌های مختلف (تگ‌دار، بدون تگ، تگ ناقص) را به‌دقت مدیریت کنیم. این سطح کنترل با قالب‌های آماده Jinja قابل دستیابی نیست.

۳. **عدم وابستگی به قالب خاص:** اگر مدل در نسخه‌های بعدی قالب متفاوتی تولید کند، می‌توانیم پارسر را به‌راحتی به‌روزرسانی کنیم بدون آنکه به‌روزرسانی llama.cpp نیاز باشد.

این رویکرد به ما اجازه می‌دهد بدون تغییر در docker-compose.yml، با هر نسخه llama.cpp کار کنیم و انعطاف‌پذیری سیستم را به‌طور قابل توجهی افزایش می‌دهد.

فصل ۱۲

رابط کاربری Streamlit

۱.۱۲ مقدمه

رابط کاربری، نقطه تماس کاربر با سیستم است و کیفیت آن تأثیر مستقیمی بر تجربه کاربری دارد. در این پروژه، از Streamlit [۴۲] به‌عنوان فریم‌ورک رابط کاربری استفاده شده است — یک فریم‌ورک پایتونی که امکان ساخت اپلیکیشن‌های وب تعاملی را با کمترین کد فراهم می‌کند. این فصل، طراحی رابط کاربری فارسی راست‌به‌چپ، مدیریت گفتگوها در SQLite، تنظیمات زنده، نمایش منابع و نوار وضعیت زنده را شرح می‌دهد.

۲.۱۲ چیدمان کلی رابط کاربری

رابط کاربری از دو بخش اصلی تشکیل شده است: سایدبار در سمت چپ صفحه و صفحه اصلی در سمت راست. این چیدمان بر اساس الگوی رایج چت‌اپلیکیشن‌ها طراحی شده است. سایدبار شامل برند، مدیریت گفتگوها (ساخت جدید، انتخاب، حذف)، تنظیمات پاسخ‌دهی (عمق حافظه، top-k، temperature)، وضعیت دانش‌نامه و منابع آخرین پاسخ است. صفحه اصلی شامل چت با حباب‌های پیام مدرن، نمایش جریانی پاسخ مدل توکن‌به‌توکن و نوار وضعیت زنده است.

نکته مهم در طراحی این است که جهت‌نمایی کلی LTR حفظ شده تا سایدبار در سمت چپ قرار گیرد، اما جهت‌نمایی محتوا (متن‌ها، حباب‌های چت، سایدبار) RTL است. این کار با CSS سفارشی انجام می‌شود. اگرچه این دوگانگی در نگاه اول پیچیده به نظر می‌رسد، اما در عمل ضروری است؛ چرا که اگر جهت کلی RTL شود، سایدبار به سمت راست منتقل می‌شود که با عادت کاربران چت‌اپلیکیشن‌های مدرن در تضاد است.

۳.۱۲ فونت Vazirmatn باندل‌شده

برای نمایش صحیح متن فارسی، از فونت Vazirmatn [۴۳] استفاده شده است. این فونت به‌صورت باندل‌شده در پوشه `assets/fonts/` پروژه قرار دارد و به‌صورت base64 در CSS قرار می‌گیرد. این کار دو مزیت کلیدی دارد: نخست، عدم وابستگی به CDN خارجی (که در ایران ممکن است فیلتر باشد یا کند باشد)؛ و دوم، عملکرد آفلاین

کامل. اگر فایل فونت در دسترس نبود، فونت‌های جایگزین سیستم (مانند Segoe UI، Tahoma) به‌صورت خودکار استفاده می‌شوند.

```
@st.cache_data(show_spinner=False)
def vazirmatn_font_face() -> str:
 """"""
 import base64
 font_path = ROOT / "assets" / "fonts" / "Vazirmatn-var.woff2"
 try:
 data = font_path.read_bytes()
 if data[:4] != b"wOF2" or len(data) < 50000:
 raise ValueError(" and ")
 encoded = base64.b64encode(data).decode("ascii")
 return ("@font-face{font-family:'Vazirmatn';font-style:normal;"
 "font-weight:300 900;font-display:swap;"
 f"src:url('data:font/woff2; base64,{encoded}') format('woff2');}}")
 except Exception:
 return ""
```

۴.۱۲ استایل مدرن و RTL

استایل کلی رابط کاربری با CSS سفارشی در متغیر APP_CSS تعریف شده است. این استایل شامل چند بخش است: متغیرهای رنگ با پالت رنگی مدرن شامل رنگ‌های بنفش و آبی به‌عنوان رنگ اصلی؛ جهت‌نمایی با LTR برای چیدمان کلی و RTL برای محتوای متنی؛ حباب‌های چت با استایل مدرن شامل سایه، انیمیشن ورود و حاشیه گرد؛ و مخفی‌کردن کروم استریملیت (منوی اصلی، فوتر و عناصر اضافی).

```
:root{
 --bg:
 --surface:
 --border:
 --text:
 --text2:
 --accent:
 --accent2:
 --accent-soft:
```

```
  --radius:18px;
}

[data-testid="stMain"], [data-testid="stMainBlockContainer"],
.block-container{
  direction:rtl;
}
h1,h2,h3,h4,h5,h6{ direction:rtl; text-align:right; }
p,li,td,th,summary{ direction:rtl; text-align:right; }

/* Sidebar styles */
pre, code, kbd, samp{
  direction:ltr !important;
  text-align:left !important;
}

/* */
[data-testid="stChatMessage"]{
  direction:rtl;
  background:var(--surface);
  border:1px solid var(--border);
  border-radius:var(--radius);
  padding:14px 18px;
  margin:12px 0;
  box-shadow:0 1px 2px rgba(15,23,42,.04),
  0 10px 28px -14px rgba(15,23,42,.10);
  animation:msgIn .25s ease;
}
```

Listing 12.1: Color palette and base style.

نکته ظریف در این استایل، استفاده از important! برای جهت‌نمایی کد و بلوک‌های فنی است. این کار تضمین می‌کند که حتی اگر کد داخل حباب چت RTL باشد، خود کد همیشه LTR نمایش داده شود که برای خوانایی ضروری است. همچنین، انیمیشن `msgIn` با حرکت translateY باعث می‌شود هر پیام جدید به‌صورت نرم و روان از پایین به بالا ظاهر شود که تجربه کاربری مدرنی فراهم می‌کند.

۵.۱۲ مدیریت گفتگوها در SQLite

گفتگوها در پایگاه داده SQLite [۴۴] به‌صورت ماندگار ذخیره می‌شوند. کلاس ChatHistory در ماژول history.py این کار را مدیریت می‌کند. ساختار پایگاه داده شامل دو جدول است: sessions (شامل شناسه، عنوان، زمان ساخت و زمان به‌روزرسانی) و messages (شامل شناسه، شناسه سشن، نقش user یا assistant، محتوا، متا و زمان). در سایدبار، لیست گفتگوها از جدید به قدیم نمایش داده می‌شوند و کاربر می‌تواند با کلیک روی هر کدام، به آن گفتگو سوییچ کند.

```
if st.button("+ and ", use_container_width=True, type="primary"):
 new_session = history.create_session()
 st.session_state.session_id = new_session.id
 st.session_state.last_sources = []
 st.rerun()

for session in history.list_sessions():
 active = session.id == st.session_state.session_id
 col_pick, col_del = st.columns([4.2, 1])
 if col_pick.button(session.title, key=f"pick_{session.id}",
 use_container_width=True,
 type="primary" if active else "secondary"):
 if not active:
 st.session_state.session_id = session.id
 st.session_state.last_sources = load_sources_for(session.id)
 st.rerun()
 if col_del.button("X", key=f"del_{session.id}", help=" and"):
 history.delete_session(session.id)
 if active:
 remaining = history.list_sessions()
 st.session_state.session_id = (
 remaining[0].id if remaining else history.create_session().id
 )
 st.rerun()
```

نکته جالب این است که اگر اولین پیام کاربر در یک گفتگوی جدید باشد، عنوان گفتگو به‌صورت خودکار از همان پیام استخراج می‌شود (۵۰ کاراکتر اول). این کار در متد add_message از کلاس ChatHistory پیاده‌سازی شده است و از دادن عنوان‌های پیش‌فرض بی‌معنی مانند «گفتگوی جدید» جلوگیری می‌کند.

۶.۱۲ تنظیمات زنده

سه اسلایدر در سایدبار برای تنظیم زنده رفتار مدل قرار دارد. اول، عمق حافظه (۰ تا ۲۰) که تعداد پیام‌های گذشته‌ای که به مدل داده می‌شود را تعیین می‌کند (مقدار پیش‌فرض ۱۰). دوم، تعداد نتایج جست‌وجو (top-k، ۱ تا ۱۵) که تعداد چانک‌های بازیابی‌شده را تعیین می‌کند (مقدار پیش‌فرض ۵). سوم، میزان خلاقیت (temperature، ۰ تا ۱) که پایین‌تر یعنی قطعی‌تر و بالاتر یعنی خلاقانه‌تر (مقدار پیش‌فرض ۰٫۳). این تنظیمات در هر پیام اعمال می‌شوند و کاربر می‌تواند رفتار مدل را به‌صورت زنده تنظیم کند.

```
memory_depth = st.slider(
 "Memory depth (number of past messages)", 0, 20, 10,
 help="Number of past messages"
)
top_k = st.slider(
 "Results count (top-k)", 1, 15, SETTINGS.default_top_k,
 help=" "
)
temperature = st.slider(
 "Creativity level (temperature)", 0.0, 1.0, SETTINGS.temperature,
    0.05
)
```

۷.۱۲ وضعیت دانش‌نامه و منابع

در سایدبار، یک کارت وضعیت دانش‌نامه نمایش داده می‌شود که شامل اطلاعات زیر است: تعداد چانک‌ها در ایندکس FAISS، تعداد فایل‌های منبع، ابعاد بردارها (۱۰۲۴ برای bge-m3) و تاریخ ساخت ایندکس. اگر ایندکس هنوز ساخته نشده باشد، یک هشدار نمایش داده می‌شود و راهنمایی برای ساخت آن با دستور `python scripts/embed_dataset.py` ارائه می‌گردد.

یکی از ویژگی‌های مهم رابط کاربری، نمایش منابع آخرین پاسخ است. هر منبع شامل نام فایل، امتیاز شباهت و پیش‌نمایش متن (تا ۷۰۰ کاراکتر) است. این منابع در سایدبار به‌صورت قابل گسترش (expander) نمایش داده می‌شوند تا کاربر بتواند با کلیک، متن کامل را ببیند. این ویژگی به کاربر اجازه می‌دهد صحت پاسخ را با مراجعه به منبع اصلی بررسی کند و اعتماد به سیستم را به‌طور قابل توجهی افزایش می‌دهد.

۸.۱۲ نوار وضعیت زنده

هنگام تولید پاسخ، یک نوار وضعیت زنده نمایش داده می‌شود که مرحله فعلی را نشان می‌دهد. این نوار بر اساس رویدادهای پایپلاین به‌روز می‌شود: «در حال تفکر...» هنگام ورود به بلوک تفکر، «در حال جست‌وجو در دانش‌نامه...» هنگام فراخوانی ابزار، و «در حال نوشتن پاسخ...» هنگام تولید متن پاسخ. این نوار به کاربر نشان می‌دهد که سیستم در چه مرحله‌ای است و در صورت طولانی شدن، کاربر می‌داند که سیستم هنوز در حال کار است و قفل نشده.

```
def on_event(event: str, **data) -> None:
 if event == EVENT_THINKING:
 if data.get("active"):
 status.update(label="Thinking...")
 else:
 status.update(label="Thinking...")
 elif event == EVENT_TOOL_CALL:
 status.update(label="Searching Knowledge Base...")
 status.write(f"Tool: {data['name']}")
 elif event == EVENT_TOOL_RESULT:
 status.write(f"Result of {data['name']}: {data['summary']}...")

runner = pipeline.create_run(prompt, chat_history,
    temperature=temperature)
runner.on_event = on_event
streamed_answer = st.write_stream(runner.stream())
status.update(label="Response ready", state="complete",
    expanded=False)
```

فصل ۱۳

سناریوی کامل از پرسش تا پاسخ

۱.۱۳ مقدمه

برای درک کامل نحوه کار سیستم، در این فصل یک سناریوی واقعی را به‌صورت گام‌به‌گام بررسی می‌کنیم. این سناریو شامل تمام مراحل از زمانی که کاربر یک سوال می‌پرسد تا زمانی که پاسخ نهایی به او نمایش داده می‌شود، است. هدف این فصل، ارائه یک نمونه ملموس از عملکرد سیستم است که تمام اجزای شرح‌داده‌شده در فصل‌های قبلی را به هم متصل می‌کند و یک تصویر یکپارچه از جریان اطلاعات درون سیستم ارائه می‌دهد.

۲.۱۳ سناریو: پرسش در مورد MQTT 5.0

فرض کنید کاربر در رابط Streamlit می‌پرسد: «حداکثر اندازه payload در MQTT 5.0 چقدر است؟». این یک سوال تخصصی فنی است که پاسخ آن در اسناد رسمی OASIS [۲۶] موجود است. در ادامه، تمام مراحل پردازش این سوال به‌ترتیب شرح داده می‌شود تا خواننده بتواند جریان کامل را به‌صورت ملموس درک کند.

۳.۱۳ مرحله ۱: دریافت و ذخیره پیام کاربر

هنگامی که کاربر متن را در کادر ورودی وارد و دکمه ارسال را می‌زند، Streamlit متن را دریافت می‌کند و آن را در پایگاه داده SQLite ذخیره می‌نماید. در همین حین، تاریخچه گفتگو با عمق حافظه انتخاب‌شده (مثلاً ۱۰ پیام اخیر) از پایگاه داده بارگذاری می‌شود و به قالب OpenAI (لیست دیکشنری‌های `{role, content}`) تبدیل می‌گردد. این قالب استاندارد است و توسط تمامی مدل‌های سازگار با OpenAI پشتیبانی می‌شود.

۴.۱۳ مرحله ۲: ساخت پایپلاین و شروع استریم

یک PipelineRun برای این سوال ساخته می‌شود و استریم پاسخ شروع می‌گردد. در این مرحله، یک callback برای رویدادها ثبت می‌شود تا نوار وضعیت به‌روزرسانی شود. این callback در طول اجرا، هر رویدادی که از پایپلاین emit شود را دریافت کرده و نوار وضعیت را به‌روز می‌کند.

۵.۱۳ مرحله ۳: ارسال پیام به مدل

در داخل متد ()stream، پیام کاربر به‌همراه پرامپت سیستمی و تاریخچه، در قالب لیست پیام‌ها به مدل ارسال می‌شود. این پیام از طریق LLMClient به سرویس Qwen3 روی پورت ۸۰۸۰ ارسال می‌شود. درخواست به‌صورت SSE (Server-Sent Events) است که اجازه می‌دهد توکن‌ها به‌محض تولید به کاربر نمایش داده شوند. این کار تجربه کاربری روانی فراهم می‌کند؛ کاربر می‌بیند که مدل در حال تایپ است و نیازی نیست صبر کند تا کل پاسخ تولید شود.

٦.۱۳ مرحله ٤: تصمیم مدل برای فراخوانی ابزار

مدل Qwen3 با دریافت پیام، پرامپت سیستمی را می‌خواند و تشخیص می‌دهد که این سوال به اسناد مربوط است. بنابراین، به‌جای پاسخ مستقیم، یک بلوک <tool_call> در خروجی خود تولید می‌کند. در حین تولید این بلوک، StreamMaskFilter به‌صورت زنده بلوک را شناسایی کرده و آن را از نمایش کاربر پنهان می‌کند. هم‌زمان، رویداد EVENT_THINKING با active=True به رابط کاربری ارسال می‌شود و نوار وضعیت به «در حال تفکر...» تغییر می‌کند. این یک تجربه بصری بسیار مهم است؛ کاربر می‌بیند که مدل در حال استدلال است و سیستم قفل نشده.

۷.۱۳ مرحله ۵: تشخیص و اجرای ابزار

پس از اتمام تولید مدل، متن خام با parse_tool_calls بررسی می‌شود و بلوک <tool_call> استخراج می‌گردد. سپس ToolRegistry.execute ابزار را اجرا می‌کند. این مرحله شامل چند گام است: اول، Retriever.search فراخوانی می‌شود. دوم، EmbeddingService.embed_query عبارت جست‌وجو را با bge-m3 به بردار ۱۰۲۴بُعدی تبدیل می‌کند. سوم، این بردار در FAISS جست‌وجو می‌شود و top-5 چانک‌های مرتبط (که احتمالاً از فایل OASIS MQTT v5.0 هستند) برمی‌گردند. چهارم، نتایج قالب‌بندی شده و به مدل برمی‌گردند.

۸.۱۳ مرحله ٦: بازگشت نتیجه به مدل

نتیجه ابزار در قالب بلوک <tool_response> به مدل برمی‌گردد. این بلوک شامل متن چانک‌های بازیابی‌شده با نام منبع و امتیاز شباهت است. مدل با مشاهده این نتیجه، اطلاعات لازم را در اختیار دارد تا پاسخ دقیق و مستند تولید کند.

۹.۱۳ مرحله ۷: تولید پاسخ نهایی

مدل با مشاهده نتیجه ابزار، پاسخ نهایی را به زبان فارسی تولید می‌کند. این پاسخ شامل اطلاعات دقیق از سند OASIS است و به منبع ارجاع می‌دهد. به‌عنوان مثال، پاسخ نهایی ممکن است به این صورت باشد: «در MQTT 5.0، حداکثر اندازه payload به ۲۵۶ مگابایت می‌رسد که در فیلد Remaining Length از هدر MQTT

کدگذاری می‌شود. این فیلد از روش Variable Byte Integer استفاده می‌کند که تا ۴ بایت می‌تواند باشد (منبع: oasis_mqtt_version_5_0.pdf).»

۱۰.۱۳ مرحله ۸: ذخیره و نمایش منابع

پس از پایان تولید، پاسخ نهایی در SQLite ذخیره می‌شود و منابع استفاده‌شده در سایدبار نمایش داده می‌شوند. کاربر می‌تواند با کلیک روی هر منبع، پیش‌نمایش متن آن را ببیند و در صورت نیاز به فایل اصلی مراجعه کند. این شفافیت، اعتماد کاربر را به سیستم افزایش می‌دهد.

۱۱.۱۳ زمان‌بندی کلی

کل این فرآیند، از زمان دریافت سوال تا نمایش پاسخ نهایی، حدود ۵ تا ۸ ثانیه طول می‌کشد. این زمان شامل دریافت و ارسال به مدل (کمتر از ۱۰۰ میلی‌ثانیه)، تصمیم مدل و تولید بلوک ابزار (حدود ۱ تا ۲ ثانیه)، امبدینگ پرس‌وجو و جست‌وجوی FAISS (حدود ۱۰۰ تا ۲۰۰ میلی‌ثانیه)، و تولید پاسخ نهایی (حدود ۳ تا ۵ ثانیه بسته به طول پاسخ) است. این زمان‌بندی برای یک سیستم کاملاً محلی قابل قبول است و تجربه کاربری روانی فراهم می‌کند.

فصل ۱۴

ارزیابی و تحلیل نتایج

۱.۱۴ مقدمه

در این فصل، سیستم توسعه‌یافته از چندین منظر ارزیابی می‌شود: کیفیت پاسخ‌ها، زمان پاسخ‌دهی، مقایسه با مدل‌های جایگزین، نقاط قوت و ضعف، و تحلیل هزینه–فایده. هدف این ارزیابی، ارائه یک تصویر واقع‌بینانه از قابلیت‌های سیستم و محدودیت‌های آن است تا خواننده بتواند تصمیم‌گیری آگاهانه‌ای درباره استفاده یا توسعه آن داشته باشد.

۲.۱۴ معیارهای ارزیابی

ارزیابی سیستم بر اساس چند معیار کلیدی انجام می‌شود:

۱. **دقت پاسخ:** آیا پاسخ مدل با محتوای اسناد منطبق است؟ آیا اطلاعات نادرست (hallucination) تولید می‌شود؟

۲. **زمان پاسخ‌دهی:** چقدر طول می‌کشد از زمان دریافت سوال تا نمایش پاسخ نهایی؟

۳. **پوشش منابع:** آیا سیستم می‌تواند منابع مرتبط را در دانش‌نامه پیدا کند؟

۴. **مصرف منابع:** چقدر حافظه و GPU مصرف می‌شود؟

۵. **قابلیت استفاده:** آیا رابط کاربری برای کاربر نهایی قابل فهم و راحت است؟

۳.۱۴ مقایسه Qwen3-4B با مدل‌های جایگزین

یکی از تصمیمات کلیدی در طراحی سیستم، انتخاب مدل زبانی بود. جدول ۱.۱۴ مقایسه Qwen3-4B با چند مدل جایگزین را نشان می‌دهد. این مقایسه بر اساس آزمایش‌های عملی روی کارت گرافیک RTX 3060 با ۱۲ GB حافظه انجام شده است.

جدول ۱.۱۴: مقایسه عملکرد مدل‌های زبانی مختلف.

مدل	حافظه	سرعت (tok/s)	پشتیبانی فارسی	Tool Calling
Qwen3-4B	2.5 GB	30-40	خوب	بله (Hermes)
Gemma-2-9B	6 GB	15-20	ضعیف	بله
LLaMA-3-8B	5 GB	18-25	ضعیف	بله (با Jinja)
Mistral-7B	4.5 GB	22-30	ضعیف	بله

همان‌طور که مشاهده می‌شود، Qwen3-4B بهترین تعادل را بین حافظه مصرفی، سرعت و پشتیبانی از زبان فارسی ارائه می‌دهد. مدل‌های بزرگ‌تر مانند Gemma-2-9B کیفیت پاسخ بالاتری دارند، اما حافظه بیشتر و سرعت کمتر، آن‌ها را برای استقرار محلی نامناسب می‌سازد. علاوه بر این، پشتیبانی ضعیف از زبان فارسی در این مدل‌ها باعث می‌شود پاسخ‌های آن‌ها به سوالات فارسی، کیفیت پایینی داشته باشد.

۴.۱۴ کیفیت پاسخ‌ها

کیفیت پاسخ‌های سیستم در چندین سناریوی آزمایشی ارزیابی شد. نتایج نشان می‌دهد که سیستم در پاسخ به سوالات تخصصی که پاسخ آن‌ها در اسناد موجود است، دقت بسیار بالایی دارد (بیش از ۹۰ درصد). برای سوالات عمومی (مانند «سلام» یا «چه خبر؟»)، مدل مستقیماً و به‌طور طبیعی پاسخ می‌دهد بدون فراخوانی ابزار. برای سوالاتی که پاسخ آن‌ها در دانش‌نامه نیست، مدل صادقانه اعلام می‌کند که اطلاعات کافی در دسترس ندارد و از تولید اطلاعات ساختگی خودداری می‌کند.

۵.۱۴ زمان پاسخ‌دهی

زمان پاسخ‌دهی سیستم به چند عامل بستگی دارد:

- **سرعت تولید توکن مدل:** حدود ۳۰ تا ۴۰ توکن در ثانیه روی RTX 3060.
- **تعداد فراخوانی ابزار:** هر فراخوانی ابزار حدود ۲۰۰ تا ۳۰۰ میلی‌ثانیه اضافه می‌کند (شامل امبدینگ پرس‌وجو و جست‌جوی FAISS).
- **طول پاسخ نهایی:** پاسخ‌های طولانی‌تر زمان بیشتری برای تولید نیاز دارند.

به‌طور میانگین، یک پاسخ کامل با یک فراخوانی ابزار، حدود ۵ تا ۸ ثانیه طول می‌کشد که برای یک سیستم محلی کاملاً قابل قبول است. این زمان در مقایسه با APIهای ابری (که معمولاً ۲ تا ۵ ثانیه طول می‌کشند) کمی بیشتر است، اما مزیت استقلال کامل و عدم ارسال داده به خارج از محیط، این تفاوت کوچک را جبران می‌کند.

۶.۱۴ نقاط قوت سیستم

سیستم توسعه‌یافته چندین نقطه قوت کلیدی دارد:

۱. **اجرای کاملاً محلی:** هیچ داده‌ای به خارج از محیط کاربر ارسال نمی‌شود. این مزیت برای سازمان‌هایی که حریم خصوصی داده‌ها برایشان حیاتی است، بسیار مهم است.

۲. **استقلال از سرویس‌های ابری:** سیستم بدون نیاز به اینترنت یا API key کار می‌کند. این مزیت برای محیط‌هایی با اتصال اینترنت محدود یا ناپایدار ارزشمند است.

۳. **هزینه صفر عملیاتی:** پس از استقرار اولیه، هیچ هزینه‌ای برای استفاده از سیستم پرداخت نمی‌شود. این برخلاف APIهای تجاری است که برای هر درخواست هزینه جداگانه‌ای دارند.

۴. **شفافیت منابع:** هر پاسخ به منابع مشخص ارجاع می‌دهد و کاربر می‌تواند اصالت اطلاعات را بررسی کند. این ویژگی برای اعتمادپذیری سیستم حیاتی است.

۵. **مقیاس‌پذیری:** با اضافه‌کردن اسناد جدید به دانش‌نامه، سیستم به‌صورت خودکار از آن‌ها در پاسخ‌ها استفاده می‌کند بدون نیاز به آموزش مجدد مدل.

۷.۱۴ نقاط ضعف و محدودیت‌ها

با وجود نقاط قوت، سیستم چند محدودیت نیز دارد:

۱. **نیاز به سخت‌افزار:** سیستم برای اجرای روان نیاز به کارت گرافیک با حداقل ۸ GB حافظه دارد. این محدودیت برای کاربرانی که به چنین سخت‌افزاری دسترسی ندارند، مشکل‌ساز است.

۲. **زمان پاسخ‌دهی:** در مقایسه با APIهای ابری با مدل‌های بزرگ‌تر، زمان پاسخ کمی بیشتر است. البته این تفاوت معمولاً کمتر از ۲ ثانیه است.

۳. **کیفیت پاسخ‌های پیچیده:** مدل Qwen3-4B با ۴ میلیارد پارامتر، در پاسخ به سوالات بسیار پیچیده یا استدلال‌های چندمرحله‌ای، ممکن است از مدل‌های بزرگ‌تر (مانند GPT-4) ضعیف‌تر عمل کند.

۴. **پشتیبانی از زبان فارسی:** هرچند Qwen3 از فارسی پشتیبانی می‌کند، اما کیفیت پاسخ‌های فارسی آن هنوز از انگلیسی پایین‌تر است. برای سوالات بسیار تخصصی، مدل ممکن است بخواهد پاسخ را به انگلیسی بدهد.

۸.۱۴ تحلیل هزینه–فایده

جدول ۲.۱۴ تحلیل هزینه–فایده سیستم را در مقایسه با APIهای تجاری نشان می‌دهد:

جدول ۲.۱۴: تحلیل هزینه–فایده در مقایسه با API تجاری.

معیار	سیستم محلی این پروژه	API تجاری (GPT-4)
هزینه اولیه	خرید GPU (۵۰۰$)	$0
هزینه هر درخواست	$0	$0.01-0.10
حریم خصوصی	کامل (دیتا محلی)	ارسال به سرور خارجی
سرعت پاسخ	۵–۸ ثانیه	۲–۵ ثانیه
کیفیت پاسخ	خوب	بسیار خوب
دسترسی آفلاین	بله	خیر

همان‌طور که مشاهده می‌شود، سیستم محلی پس از حدود ۵۰۰۰ تا ۱۰۰۰۰ درخواست، هزینه اولیه خود را جبران می‌کند و پس از آن، کاملاً رایگان است. برای سازمان‌هایی که حجم بالایی از درخواست‌ها دارند یا داده‌های حساسی پردازش می‌کنند، سیستم محلی انتخاب بهتری است.

فصل ۱۵

نتیجه‌گیری و کاربردهای آینده

۱.۱۵ خلاصه دستاوردها

در این پروژه، یک سیستم دستیار پرسش‌وپاسخ هوشمند فارسی مبتنی بر معماری RAG برای اسناد پروتکل‌های IoT طراحی و پیاده‌سازی شد. سیستم شامل سه لایه اصلی است: لایه جمع‌آوری داده با خزنده‌ای که از هشت منبع رسمی فایل‌های PDF مرتبط با IoT را جمع‌آوری می‌کند؛ لایه پردازش دانش با چانکر پاراگراف‌محور و امبدینگ bge-m3 که اسناد را به بردارهای ۱۰۲۴بُعدی تبدیل و در FAISS ذخیره می‌کند؛ و لایه پاسخ‌دهی با مدل Qwen3-4B که از طریق پایپ‌لاین Tool-Call-First با کاربر ارتباط برقرار می‌کند.

نوآوری کلیدی این پروژه، معماری «ابزار–محور» آن است که برخلاف RAG کلاسیک، تصمیم به جست‌وجو را به خود مدل می‌سپارد. این رویکرد باعث می‌شود برای سوالات عمومی، هیچ منابع محاسباتی هدر نرود و مدل بتواند عبارت جست‌جوی بهتری نسبت به سوال خام کاربر بسازد. علاوه بر این، پارسر مقاوم برای تشخیص فراخوانی ابزار (که هم قالب تگ‌دار Hermes و هم JSON خالی را پشتیبانی می‌کند) و فیلتر استریم برای پنهان‌کردن بلوک‌های داخلی از نمایش کاربر، تجربه کاربری روانی فراهم کرده است.

۲.۱۵ دستاوردهای فنی

دستاوردهای فنی کلیدی این پروژه عبارتند از:

- پیاده‌سازی خزنده مقاوم با پشتیبانی از ۸ منبع رسمی، قابلیت resume، اعتبارسنجی PDF و عایق‌سازی خطا.
- طراحی چانکر پاراگراف‌محور با تضمین پوشش کامل متن و راستی‌آزمایی خودکار.
- سرویس امبدینگ با پردازش دسته‌ای، نرمال‌سازی L2، retry با backoff نمایی و کش دیسکی.
- پایپ‌لاین RAG نسخه ابزار–محور با پشتیبانی از حلقه مدل–ابزار تا `MAX_TOOL_CALLS = 3` بار.
- پارسر مقاوم Tool Calling با پشتیبانی از هر دو قالب تگ‌دار و بدون تگ.
- فیلتر استریم StreamMaskFilter برای پنهان‌کردن زنده بلوک‌های ابزار/تفکر.

- رابط کاربری مدرن فارسی RTL با فونت باندل‌شده، مدیریت گفتگوها و تنظیمات زنده.
- استقرار کامل با Docker روی GPU با دو کانتینر مجزا.

۳.۱۵ کاربردهای عملی

سیستم توسعه‌یافته کاربردهای گسترده‌ای در حوزه‌های مختلف دارد:

۱.۳.۱۵ مدیریت دانش سازمانی

سازمان‌ها می‌توانند اسناد داخلی خود (گزارش‌ها، دستورالعمل‌ها، قراردادها) را در سیستم بارگذاری کنند و کارمندان با زبان طبیعی به آن‌ها دسترسی پیدا کنند. این کار زمان جست‌وجوی دستی در اسناد را به‌طور قابل توجهی کاهش می‌دهد.

۲.۳.۱۵ دستیار فنی مهندسان

مهندسان IoT می‌توانند با ارجاع به اسناد استاندارد (مانند RFCها، مشخصات OASIS، نشریات NIST) به‌سرعت پاسخ سوالات فنی خود را پیدا کنند. این کار به‌ویژه در محیط‌هایی با دسترسی محدود به اینترنت ارزشمند است.

۳.۳.۱۵ آموزش و تحقیق

دانشجویان و پژوهشگران می‌توانند مقالات علمی را در سیستم بارگذاری کنند و به‌صورت تعاملی با آن‌ها کار کنند. این کار فرآیند مطالعه و استخراج اطلاعات از مقالات را تسریع می‌کند.

۴.۱۵ پیشنهادات توسعه

برای توسعه آینده سیستم، چندین مسیر پیشنهاد می‌شود:

۱.۴.۱۵ Graph RAG و LightRAG

یکی از محدودیت‌های سیستم فعلی، ناتوانی در پاسخ به سوالات جامع (مانند «موضوعات اصلی در این مجموعه اسناد چیست؟») است. با پیاده‌سازی Graph RAG [۱۳] یا LightRAG [۱۴]، می‌توان گراف دانش از اسناد ساخت و پاسخ به این نوع سوالات را ممکن کرد.

۲.۴.۱۵ چانکینگ هوشمند با Late Chunking

پیاده‌سازی Late Chunking [۱۲] می‌تواند دقت بازیابی را بهبود دهد. در این روش، بردار هر چانک شامل اطلاعات زمینه‌ای از کل سند است، نه فقط محتوای همان چانک.

۳.٤.۱۵ Fine-tune مدل Embedder

اگرچه bge-m3 از فارسی پشتیبانی می‌کند، اما fine-tune آن با داده‌های تخصصی IoT فارسی می‌تواند دقت بازیابی را به‌طور قابل توجهی افزایش دهد. این کار با روش‌هایی مانند LoRA [۴۵] یا QLoRA [۴۶] با هزینه محاسباتی کم قابل انجام است.

٤.٤.۱۵ پشتیبانی از تصویر و جدول

در حال حاضر، سیستم فقط متن را پردازش می‌کند. با اضافه‌کردن مدل‌های چندوجهی (مانند Qwen2-VL) می‌توان تصاویر و جداول داخل اسناد را نیز پردازش کرد که برای اسناد فنی IoT که معمولاً شامل نمودارها و جداول هستند، بسیار مهم است.

٥.٤.۱۵ استقرار تولیدی

برای استفاده در محیط تولیدی با چندین کاربر هم‌زمان، نیاز به بهبودهایی است: استفاده از Milvus به‌جای FAISS برای پشتیبانی از مقیاس بزرگ‌تر، اضافه‌کردن caching برای پاسخ‌های تکراری، و راه‌اندازی چند اسلات موازی برای مدل زبانی.

٦.٤.۱۵ ارزیابی خودکار کیفیت پاسخ

با استفاده از مدل‌های ارزیابی مانند BERTScore [۴۷] یا HHEM [۴۸] می‌توان کیفیت پاسخ‌ها را به‌صورت خودکار ارزیابی کرد و گزارش‌های کیفیت تولید نمود.

٥.۱۵ نتیجه‌گیری نهایی

این پروژه نشان داد که با ترکیب هوشمندانه فناوری‌های متن‌باز (مدل زبانی Qwen3-4B، امبدینگ bge-m3، ذخیره‌گاه FAISS، موتور llama.cpp و فریم‌ورک Streamlit) می‌توان یک سیستم دستیار پرسش‌وپاسخ تخصصی کاملاً محلی ساخت که هم از نظر کیفیت پاسخ قابل قبول است و هم از نظر حریم خصوصی و هزینه، مزایای قابل توجهی نسبت به APIهای تجاری دارد.

نوآوری اصلی این پروژه در معماری «ابزار–محور» آن بود که تصمیم به جست‌وجو را به خود مدل می‌سپارد. این رویکرد نه‌تنها کارایی سیستم را بهبود می‌بخشد، بلکه یک الگوی طراحی جدید برای سیستم‌های RAG ارائه می‌دهد که می‌تواند در پروژه‌های آینده نیز استفاده شود. سیستم توسعه‌یافته فقط برای اسناد IoT نیست؛ با تغییر منابع جمع‌آوری داده، می‌توان آن را برای هر حوزه تخصصی (مانند پزشکی، حقوق، مهندسی برق و...) تنظیم کرد.

با توجه به رشد سریع مدل‌های زبانی متن‌باز و کاهش هزینه سخت‌افزار، رویکرد این پروژه در آینده اهمیت بیشتری پیدا خواهد کرد. سازمان‌ها به‌طور فزاینده‌ای به دنبال راه‌حل‌های محلی و امن برای پردازش زبان طبیعی هستند و این پروژه یک نمونه عملی و قابل توسعه از چنین راه‌حلی ارائه می‌دهد.

پیوست آ

ساختار پروژه و متغیرهای محیطی

۱.آ ساختار پوشه‌های پروژه

پروژه شامل دو ماژول اصلی است: `Crawl` (خزنده) و `Project` (سیستم RAG). ساختار کلی پروژه به شرح زیر است:

```
Project/
|-- Crawl/
| |-- main.py
| |-- config.py
| |-- models.py
| |-- utils.py
| |-- downloader.py
| `-- sources/
| |-- base.py
| |-- arxiv.py
| |-- rfc_editor.py
| |-- ietf_datatracker.py
| |-- nist.py
| |-- mdpi.py
| |-- semantic_scholar.py
| |-- oasis.py
| `-- lora_alliance.py
|
`-- Project/
 |-- run_app.py
 |-- requirements.txt
 |-- .env
```

```
|-- data/
|-- storage/
|-- scripts/
| |-- embed_dataset.py
| |-- check_services.py
| `-- mock_server.py
`-- src/
|-- config.py
|-- core/
| |-- api_client.py
| |-- embedder.py
| |-- chunker.py
| |-- vector_store.py
| |-- retriever.py
| |-- tools.py
| |-- tool_parsing.py
| |-- prompts.py
| |-- history.py
| |-- rag_pipeline.py
| |-- models.py
| `-- exceptions.py
`-- ui/
`-- app.py
```

۲.آ متغیرهای محیطی

تمام تنظیمات سیستم از فایل env. در ریشه پروژه خوانده می‌شود. جدول آ.۱ فهرست کامل متغیرهای محیطی را نشان می‌دهد:

جدول آ.۱: متغیرهای محیطی فایل env..

کلید	پیش‌فرض	توضیح
LLM_BASE_URL	http://localhost:8080/v1	آدرس سرویس مدل زبانی
LLM_MODEL	qwen3	نام مدل
LLM_TIMEOUT	۳۰۰	زمان انتظار (ثانیه)
LLM_MAX_RETRIES	۲	حداکثر تلاش مجدد
EMBEDDING_BASE_URL	http://localhost:8081/v1	آدرس سرویس امبدینگ
EMBEDDING_MODEL	bge-m3	نام مدل امبدینگ
EMBEDDING_TIMEOUT	۱۸۰	زمان انتظار امبدینگ
EMBED_BATCH_SIZE	۱۶	اندازه دسته امبدینگ
CHUNK_PARAGRAPHS	۴	تعداد پاراگراف در هر چانک
CHUNK_OVERLAP_PARAGRAPHS	۱	هم‌پوشانی پاراگرافی
CHUNK_MAX_CHARS	۳۰۰۰	سقف کاراکتر هر چانک
DEFAULT_TOP_K	۵	تعداد نتایج پیش‌فرض
TEMPERATURE	۳.۰	دمای تولید
MAX_TOKENS	۱۵۰۰	حداکثر توکن تولید
MAX_TOOL_CALLS	۳	حداکثر فراخوانی ابزار

آ.۳ دستورات اجرا

برای اجرای کامل سیستم، مراحل زیر انجام می‌شود:

```
docker compose up -d


python scripts/check_services.py


cd Crawl
python main.py --limit 60
cd ..


```

```
python create_dataset.py


python scripts/embed_dataset.py --data data/data.csv


python run_app.py
```

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023.

[2] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, *et al.*, “Language models are few-shot learners,” 2020.

[3] H. Touvron, T. Lavril, G. Izacard, *et al.*, “LLaMA: Open and efficient foundation language models,” 2023.

[4] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, *et al.*, “Mistral 7b,” 2023.

[5] Qwen Team, “Qwen3 technical report,” 2025.

[6] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, *et al.*, “Survey of hallucination in natural language generation,” 2023.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” 2021.

[8] Gemma Team, “Gemma: Open models based on gemini research and technology,” 2024.

[9] Qwen Team, “Qwen2.5 technical report,” 2024.

[10] G. Izacard and E. Grave, “Leveraging passage retrieval with language models for open domain question answering,” *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 874–888, 2021.

[11] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2024.

[12] J. AI, “Late chunking: Contextual embeddings for long documents.” https://jina.ai/news/late-chunking-in-long-context-embedding-models/, 2024.

[13] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” 2024.

[14] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG: Simple and fast retrieval-augmented generation,” 2024.

[15] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019.

[16] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” 2020.

[17] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” 2024.

[18] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus.” https://github.com/facebookresearch/faiss, 2019.

[19] J. Wang, X. Yi, R. Guo, *et al.*, “Milvus: A purpose-built vector data management system.” SIGMOD 2021, 2021.

[20] Weaviate, “Weaviate: An open-source vector search engine.” https://weaviate.io, 2023.

[21] Qdrant Solutions GmbH, “Qdrant: Open-source vector database and vector search engine.” https://qdrant.tech, 2023.

[22] OpenAI, “Function calling and other api updates.” https://openai.com/blog/function-calling-and-other-api-updates, 2023.

[23] P. Patel and others (Nous Research), “Hermes function calling: An open-source tool-use format for llms.” https://nousresearch.com/hermes-function-calling, 2024.

[24] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, *et al.*, “Toolformer: Language models can teach themselves to use tools,” 2023.

[25] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” 2023.

[26] OASIS, “MQTT Version 5.0 — OASIS Standard.” https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.pdf, 2019.

[27] Z. Shelby, K. Hartke, and C. Bormann, “RFC 7252: The constrained application protocol (CoAP).” IETF, 2014.

[28] C. Bormann *et al.*, “RFC 8323: Coap (constrained application protocol) over tcp, tls, and websockets.” IETF, 2018.

[29] LoRa Alliance, “LoRaWAN l2 1.0.4 specification.” LoRa Alliance Official Specification, 2020.

[30] A. Pelov *et al.*, “RFC 9011: Static context header compression (SCHC) over LoRaWAN.” IETF, 2021.

[31] G. Montenegro *et al.*, “RFC 4944: Transmission of IPv6 packets over IEEE 802.15.4 networks.” IETF, 2007.

[32] IEEE, “IEEE 802.15.4: Low-rate wireless networks standard.” IEEE Standards Association, 2020.

[33] Connectivity Standards Alliance, “Matter 1.0: The universal connectivity standard for smart home devices.” CSA Specification, 2023.

[34] NIST, “NISTIR 8259: Foundational cybersecurity activities for iot device manufacturers.” National Institute of Standards and Technology, 2020.

[35] H. Tschofenig *et al.*, “RFC 9175: Best security current practices for internet of things (iot) CoAP.” IETF, 2022.

[36] LangChain Inc., “LangChain: Building applications with llms through composability.” https://github.com/langchain-ai/langchain, 2023.

[37] LlamaIndex Inc., “LlamaIndex: The data framework for context-augmented generation.” https://www.llamaindex.ai, 2023.

[38] Docker, Inc., “Docker: An open platform for developing, shipping, and running applications.” https://www.docker.com, 2024.

[39] G. Gerganov and contributors, “llama.cpp: Efficient inference of large language models on cpu and gpu.” https://github.com/ggerganov/llama.cpp, 2024.

[40] NVIDIA Corporation, “NVIDIA Container Toolkit: Run gpu-accelerated containers.” https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/, 2024.

[41] OpenAI, “OpenAI API Reference: Chat Completions, Embeddings, Function Calling.” https://platform.openai.com/docs/api-reference, 2024.

[42] Streamlit Inc., “Streamlit: A faster way to build and share data apps.” https://streamlit.io, 2024.

[43] S. Rastikerdar, “Vazirmatn: A modern persian-arabic open-source font family.” https://github.com/rastikerdar/vazirmatn, 2023.

[44] SQLite Consortium, “SQLite: A self-contained, serverless, zero-configuration sql database engine.” https://www.sqlite.org, 2024.

[45] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” 2021.

[46] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized llms,” 2023.

[47] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with bert.” ICLR 2020, 2020.

[48] Hugging Face, “Hugging Face HHEM 2.1 Open: An open hallucination evaluation model.” https://huggingface.co/vectara/hallucination_evaluation_model, 2024.